

Relatório Técnico do Projeto da Unidade Curricular: Fundamentos de Desenvolvimento de Software

Diogo Ribeiro 1250871, João Bessa 1251089, Manuel Torre 1251209,
Shantall Camacho 1251460
Grupo 5, Turma 1DA

Resumo. Este artigo apresenta o desenvolvimento de um Sistema de Gestão de Planos de Viagem (SGPV), implementado em C++ e baseado na arquitetura Model-View-Controller (MVC), assegurando uma separação clara entre a lógica de negócio, a interface de utilizador e o controlo de fluxo. O sistema recorre a herança e polimorfismo através da classe base *Localidade*, que suporta a gestão de países, cidades e atividades. Permite ao utilizador registar-se, explorar um catálogo dinâmico carregado a partir de ficheiros externos e construir roteiros personalizados com validação automática de datas e horários. A aplicação evidencia robustez e escalabilidade através da adoção dos princípios SOLID e dos padrões GRASP, incluindo classes de fabrico puro para cálculos financeiros e um controlo rigoroso de exceções. A solução final automatiza o planeamento logístico e a emissão de faturas com cálculo de IVA.

Palavras chave: C++ (POO), Arquitetura MVC, Princípios SOLID, Padrões GRASP, Polimorfismo, Herança, Gestão de Itinerários, Validação de Dados, Persistência de Ficheiros, Tratamento de Exceções.

1 Introdução

O trabalho incide sobre o desenvolvimento de uma aplicação destinada à gestão de viagens. Esta aplicação será utilizada pelos colaboradores de uma agência de viagens, sendo que a cada viagem estará associado um organizador, isto é, um elemento do grupo (constituído por pelo menos um participante) responsável pela coordenação de todos os detalhes da mesma.

O organizador poderá criar e gerir viagens, definindo informações como o título, os países e as cidades a visitar, bem como as atividades a realizar em cada local selecionado.

A gestão de reservas constitui igualmente uma funcionalidade central da aplicação, permitindo registar dados como a data da reserva, o método de pagamento e o número identificador associado. Deste modo, assegura-se um controlo eficaz de todo o processo de contratação dos serviços turísticos disponibilizados pela agência.

1.1 A Digitalização do Ecossistema Turístico

No paradigma atual da Sociedade, o setor do turismo desde, há alguns anos, tem vindo a atravessar uma metamorfose digital o que tem levado a surgir certas necessidades para priorizar a sua sobrevivência. A transição dos métodos convencionais de reserva para sistemas de planeamento autónomos não passa apenas por uma questão de conveniência, mas já por uma exigência de eficiência operacional e integridade de dados. O planeamento de uma jornada contemporânea envolve a intersecção de múltiplas variáveis: restrições temporais, flutuações orçamentais, requisitos logísticos de transporte e a gestão dos perfis dos respetivos utilizadores.

Neste cenário, surge a necessidade de ferramentas que não se limitem ao armazenamento, mas que atuem como sistemas de apoio à decisão, permitindo que o organizador simule e valide roteiros complexos antes da sua execução.

1.2 A Problemática e a Relevância do Software

O desenvolvimento de um Sistema de Gestão e Planeamento de Viagens (SGPV) enfrenta desafios clássicos da Engenharia de Software. A principal problemática reside na modelação da complexidade: como representar, de forma coerente e escalável, a relação entre entidades geográficas estáticas (Países e Cidades), entidades lógicas dinâmicas (Roteiros, Agendas e Faturas) e entidades biográficas (organizador, viajante)? A ausência de um sistema integrado resulta frequentemente em redundância de dados e erros de cálculo financeiro, associação ou contagem. Assim, este projeto justifica-se pela procura de uma solução que aplique o rigor da

Programação Orientada a Objetos (POO) para mitigar estas falhas, garantindo que cada componente do sistema tenha uma responsabilidade única e bem definida.

1.3 Objetivos e Âmbito do Projeto

O objetivo central deste trabalho consiste no desenvolvimento de uma aplicação robusta em C++ que funcione como uma plataforma integrada para todo o ciclo de vida de uma viagem. Neste contexto, pretende-se, em primeiro lugar, realizar a abstração e modelação do sistema através da criação de uma hierarquia de classes baseada em herança e polimorfismo, capaz de representar de forma rigorosa a estrutura de um catálogo global. Paralelamente, visa-se assegurar uma gestão eficiente do fluxo lógico mediante a implementação de uma camada de controlo composta por múltiplos controladores especializados, garantindo que a lógica de negócio, como a validação de datas ou o cálculo de IVA, permanece devidamente isolada da interface de utilizador.

Adicionalmente, o sistema deverá suportar mecanismos de persistência heterogénea, através da implementação de processos de serialização que permitam a leitura de dados a partir de ficheiros de texto e a escrita de informação relativa a históricos e viagens, assegurando assim a continuidade e consistência dos dados. Por fim, pretende-se garantir a robustez da aplicação através de um sistema centralizado de gestão de exceções, promovendo a sua resiliência face a situações imprevistas.

1.3.1 Classificação Tripartida de Entidades

Para que o SGPV funcione com integridade, a arquitetura de dados foi estruturada em torno de três eixos fundamentais. Em primeiro lugar, consideram-se as entidades geográficas, de natureza estática, que representam o referencial físico do sistema, como é o caso das classes País e Cidade. Estas entidades são designadas como estáticas por existirem previamente à execução do programa e por os seus atributos, como o nome ou o identificador, raramente sofrerem alterações ao longo do ciclo de vida de uma viagem.

Em segundo lugar, surgem as entidades de ator ou biográficas, que correspondem às classes Organizador e Viajante. Estas representam os sujeitos da ação e distinguem-se por possuírem atributos de carácter pessoal, como o nome, a idade ou as credenciais de acesso. Assumem um papel central na estrutura do sistema, uma vez que é em torno destas entidades que a viagem é construída. No caso particular do Viajante, a sua modelação é especialmente relevante para a lógica de negócio, já que determinados atributos, como a idade, podem condicionar regras de validação, nomeadamente a participação em determinados grupos ou atividades que exijam uma idade mínima.

Por fim, destacam-se as entidades lógicas e transacionais, de natureza dinâmica, como Viagem, Agenda e Fatura. Estas não existem de forma independente, resultando da interação entre um sujeito, que realiza a viagem, e o referencial geográfico, para onde se desloca. São consideradas dinâmicas por serem criadas, alteradas e concluídas de acordo com as interações do utilizador com o sistema.

1.4 Metodologia de Desenvolvimento e Arquitetura

A metodologia adotada baseia-se no princípio da Modularidade. Utilizou-se a linguagem C++ pela sua eficiência na gestão de memória e pelo suporte nativo a tipos de dados abstratos. A arquitetura foi desenhada seguindo o padrão MVC (Model-View-Controller), com uma ênfase particular nos princípios GRASP (General Responsibility Assignment Software Patterns). O nosso trabalho visa reduzir o acoplamento(coupling) entre as funcionalidades de checkout, planeamento e catálogo, facilitando futuras expansões do sistema, como a integração com APIs externas ou interfaces gráficas.

2 Características do Sistema

A aplicação foi desenhada para operacionalizar o planeamento, orçamentação e gestão de viagens através da catalogação estruturada de destinos e do acompanhamento logístico de roteiros. O comportamento do sistema e o seu modelo de negócio são delimitados pelas seguintes características e regras de integridade:

2.1 Modelização do Domínio e Estrutura de Dados

Entidades do Domínio: O ecossistema da aplicação é centralizado em conceitos de dados fundamentais espelhados na camada Model: o RoteiroModel (entidade agregadora global do plano de viagem) e a LocalidadeModel, uma classe polimórfica

que serve de base e se especializa em PaisModel, CidadeModel e AtividadeModel (armazenando os detalhes geográficos, demográficos e de custo). O modelo integra ainda entidades de suporte encapsuladas, como o Titulo_notas_modelo e o alt_estados_modelo.

Relações de Multiplicidade: O modelo conceptual dita que a interação entre as entidades do domínio obedece a regras claras de agregação e composição. Um RoteiroModel pode conter múltiplas instâncias derivadas de LocalidadeModel associadas (sejam elas Países, Cidades ou Atividades). Adicionalmente, cada RoteiroModel possui obrigatoriamente um identificador único de reserva (n_reserva_modelo), um estado atual e um conjunto de notas associadas.

2.2 Interação e Perfis de Utilização

O sistema expõe funcionalidades de planeamento através da interface em linha de comandos (CLI), centrando-se nas interações do utilizador. Este interage com as Views (ex: Titulo_notas_view) para efetuar login, explorar o catálogo dinâmico de destinos e construir o seu roteiro personalizado. O RoteiroController atua como maestro destas ações, processando os pedidos de adição/remoção de atividades e a gestão de anotações. O processo culmina na emissão de faturas detalhadas, onde o sistema simula o cálculo automático de custos e a aplicação de impostos (IVA).

2.3 Regras de Negócio e Ciclo de Vida

Invariantes de Domínio (Restrições de Validação): Para impedir a entrada de dados inconsistentes na memória RAM e precaver falhas no sistema, o código valida ativamente os limites lógicos do negócio recorrendo ao lançamento de exceções dedicadas:

Consistência Temporal: As datas introduzidas sofrem validações rigorosas; cronologias ilógicas disparam exceções específicas como a DataInvalidaException.

Limites de Input: O título e as notas de viagem estão confinados a um intervalo estrito de tamanho (com um mínimo de 3 e um limite de 499 caracteres), rejeitando strings que excedam a capacidade planeada através de bloqueios na validação lógica.

Valores Financeiros: Não é permitida a inserção de atividades ou orçamentos com custos negativos.

Consistência e Ciclo de Vida: O sistema garante a integridade estrutural do plano de viagem. A agregação de atividades no roteiro implica um acoplamento direto à classe de fabrico puro Financeiro: qualquer alteração nas paragens da viagem despoleta o recálculo dinâmico do valor total por esta classe especializada. O ciclo de vida da viagem é, assim, ditado de forma estrita pela transição lógica do seu Estado.

2.4 Persistência Abstrata e Tolerância a Falhas

O estado da aplicação é preservado de forma não-volátil através do mapeamento para ficheiros de texto simples (.txt). Esta abordagem garante a retenção de dados cruciais,

como as credenciais de acesso avaliadas pelo LoginRepository (em dados_login.txt) e o catálogo imutável. O sistema assegura tolerância a falhas na leitura, lançando a FicheiroNaoEncontradoException caso os ficheiros de suporte sejam corrompidos ou eliminados acidentalmente.

3 Arquitetura e Design do Sistema

A conceção do SGPV foi orientada pelos princípios da robustez e da separação de interesses. A arquitetura não se limita a organizar o código, mas define como a informação flui e como o sistema se comporta perante falhas. Vemos esta arquitetura entre a figura 1 e 2.

3.1 Diagrama de Implantação: Arquitetura física

O sistema adota uma arquitetura do tipo *software standalone*, funcionando como um produto autónomo. Ao contrário das soluções baseadas na nuvem, o SGPV é auto-suficiente no ambiente onde é executado.

O processamento decorre localmente, sendo o próprio hardware responsável por todo o ciclo de vida da aplicação. A execução ocorre de forma síncrona, o que assegura que as operações de escrita em ficheiros não estão sujeitas a atrasos associados a comunicações de rede.

Relativamente à gestão de artefactos, o programa compilado interage diretamente com o sistema de ficheiros através de fluxos de entrada e saída (*fstream*). Esta abordagem garante portabilidade, permitindo que o sistema seja transferido entre diferentes diretórios sem comprometer a sua integridade, desde que a estrutura de caminhos para os ficheiros de dados seja preservada.

Por fim, o sistema distingue dois tipos de persistência: a configuração, associada ao catálogo de dados imutáveis, e a sessão, correspondente ao histórico de viagens, que é passível de alteração. Esta separação lógica contribui para prevenir a corrupção de dados durante as operações de escrita associadas à atividade do utilizador.

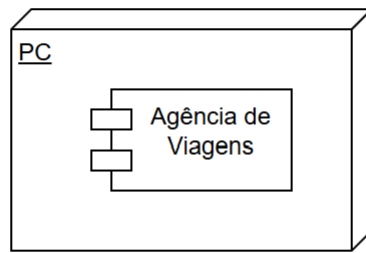


Fig. 1. Arquitetura física 1.

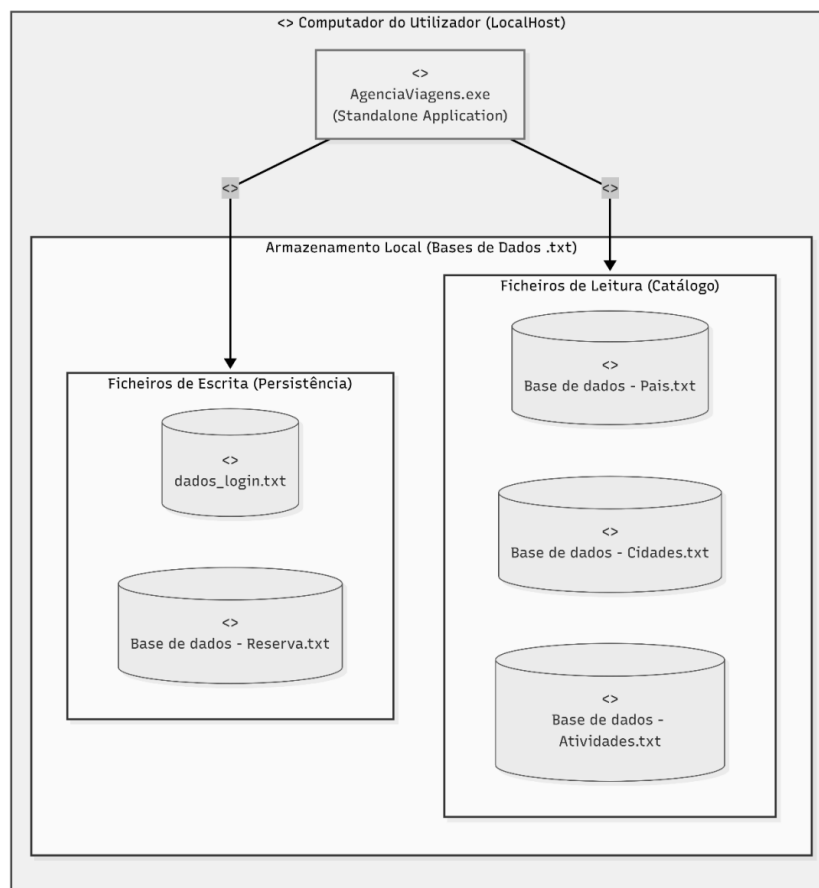


Fig. 2. Arquitetura física 2.

3.2 Diagrama de Componentes: Arquitetura lógica

Como está representado na figura 3, no que respeita ao subsistema de controlo, a lógica foi distribuída por vários controladores, evitando o anti padrão *God Object*, no qual uma única entidade concentra responsabilidades excessivas. Cada controlador assume um papel especializado, funcionando como perito numa área específica do domínio. A comunicação entre estes componentes faz-se através da troca de mensagens e referências, permitindo uma orquestração equilibrada do sistema. Por exemplo, o *CheckoutController* obtém informação junto do *GestaoCarrinhoController*, processa

esses dados e, posteriormente, invoca o *FaturaController*. Esta sequência assegura uma distribuição adequada de responsabilidades e uma maior modularidade.

Relativamente ao domínio de dados, este assenta numa hierarquia de herança. A classe abstrata *Localidade* define uma interface comum, com operações como *obterID()* e *obterPreco()*, sendo posteriormente especializada pelas classes *Pais* e *Cidade*. Esta estrutura permite que componentes como o *CatalogoController* tratem diferentes tipos de destino de forma uniforme, recorrendo ao polimorfismo.

Por fim, no que diz respeito a preocupações transversais (*cross-cutting concerns*), destaca-se o *LoggerController*, que atua como um componente independente das restantes camadas. A sua função consiste em monitorizar e registar eventos e erros provenientes de diferentes partes do sistema, como falhas na persistência de dados ou inconsistências no modelo, por exemplo datas inválidas ou problemas associados às entidades. Esta centralização permite assegurar um ponto único de auditoria e controlo de erros, contribuindo para a robustez global da aplicação.

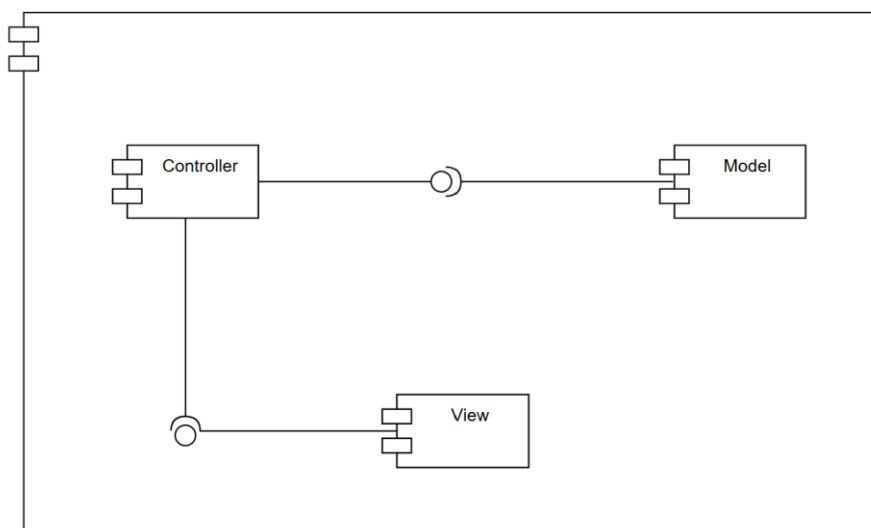


Fig. 3. Arquitetura lógica.

3.3 Árvore de UI: Design de Interação e Máquina de Estados

A Interface de Utilizador (UI) foi concebida sob a forma de uma árvore hierárquica de estados, conforme ilustrado na Figura 4. Cada nó desta estrutura corresponde a um estado do sistema no qual apenas determinadas operações se encontram disponíveis.

Numa fase inicial, o sistema encontra-se num estado de pré-autenticação, caracterizado por um acesso restrito. A transição para o menu principal apenas é permitida após validação bem-sucedida pelo *LoginController*.

No que diz respeito ao contexto de planeamento, o processo de criação de uma viagem foi definido como um fluxo sequencial, que compreende a introdução do título, a definição das datas e, posteriormente, a seleção dos destinos. Esta abordagem visa impedir a criação de viagens incompletas, garantindo a consistência dos dados desde a sua introdução.

Por outro lado, ao aceder ao histórico de viagens, o utilizador entra num modo de edição transaccional. Através do *EditorViagemController*, é possível alterar o estado de uma entidade *Viagem* em memória antes de confirmar a sua gravação definitiva. Este mecanismo aproxima-se do conceito de *commit* em sistemas de bases de dados, assegurando controlo e integridade na perspetiva de persistência dos dados.

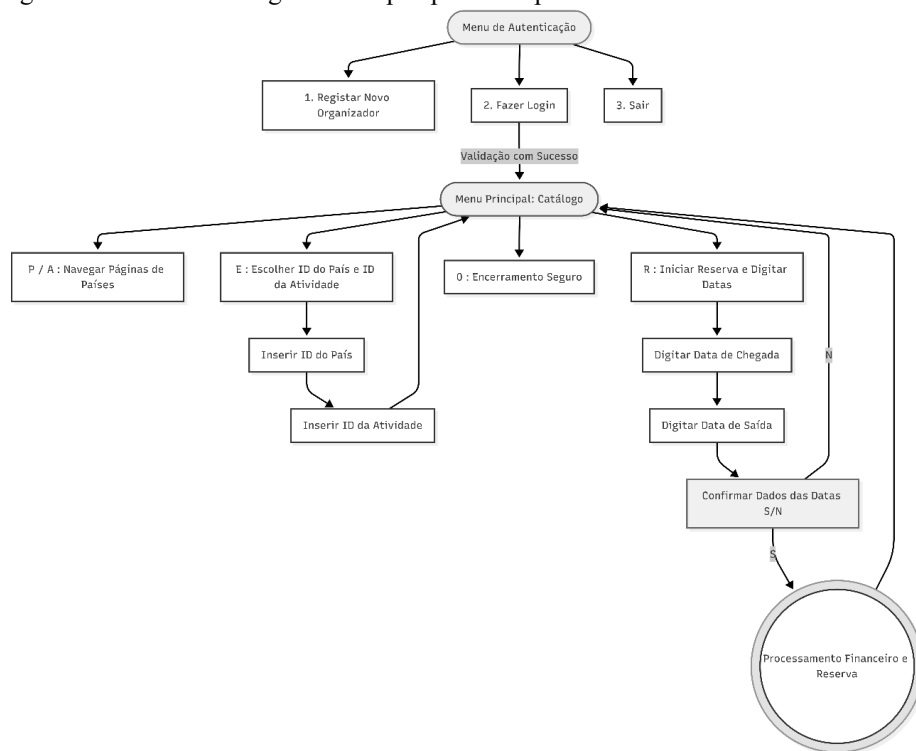


Fig. 4. Árvore UI.

4 Diagrama de classes

O Diagrama de Classes apresentado constitui a espinha dorsal estrutural do Sistema de Gestão de Planos de Viagem (SGPV). Este modelo descreve a organização estática da aplicação, detalhando as entidades, os seus atributos, métodos e, fundamentalmente, as relações de hierarquia e associação que permitem a persistência e manipulação dos

dados. A nossa arquitetura baseia-se em quatro pilares principais, visíveis na organização do diagrama.

4.1 Estrutura geral do sistema

A Figura 5 ilustra a estrutura global do sistema, evidenciando a organização dos principais componentes e as relações estabelecidas entre as diferentes classes. No centro da arquitetura destaca-se a classe *Viagem*, que funciona como o elemento agregador de toda a informação, mantendo uma relação de composição com a classe *Roteiro*, responsável pela gestão de um conjunto de objetos do tipo *Localidade*. Esta estrutura permite construir, de forma dinâmica, viagens compostas por vários destinos e atividades. A hierarquia de dados do catálogo assenta na classe abstrata *Localidade*, da qual derivam as classes *Pais*, *Cidade* e *Atividade*. Estas subclasses herdam atributos comuns, como o identificador e o nome, implementando simultaneamente comportamentos específicos, nomeadamente através do método virtual *obterPreco()*. A gestão destas entidades é assegurada pela classe *RepositorioCatalogo*, que estabelece a ligação entre o armazenamento em ficheiro e os dados mantidos em memória. No que se refere à gestão de utilizadores, observa-se uma organização baseada na classe *Pessoa*, especializada em *Organizador* e *Viajante*. A autenticação e persistência das credenciais ficam a cargo da classe *Login*, assegurando a associação entre cada viagem e o respetivo organizador. Por fim, a arquitetura integra componentes transversais que garantem o correto funcionamento do sistema, incluindo módulos dedicados ao tratamento temporal, responsáveis pela validação de datas e cálculo de durações, bem como um componente financeiro que efetua cálculos de custos e aplicação de IVA com base no roteiro definido. Adicionalmente, é utilizada uma classe de exceção personalizada, que assegura a deteção e tratamento consistente de erros associados à introdução de dados.

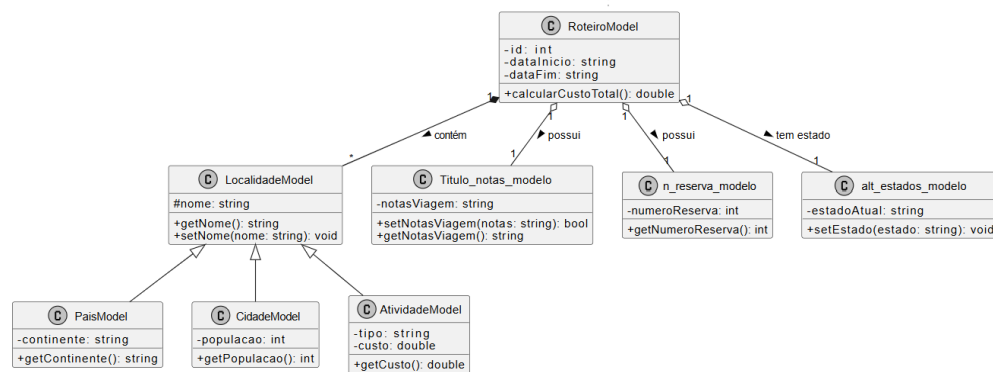


Fig. 5. Diagrama de classes.

5 Diagrama de exceções

O Diagrama de Classes de Exceções ilustra a arquitetura de robustez do SGPV. Em vez de permitir que o programa termine abruptamente perante dados inválidos ou erros de lógica, o sistema utiliza uma hierarquia de classes especializada para capturar, tratar e reportar anomalias. Esta estrutura garante que o utilizador receba feedback claro e que a integridade dos dados na memória seja preservada.

5.1 Hierarquia de Exceções

A Figura 6 apresenta a hierarquia de exceções implementada no sistema, evidenciando uma estrutura organizada e alinhada com as boas práticas da linguagem C++. No topo encontra-se a classe padrão *std::exception*, que serve de base a todo o mecanismo de tratamento de erros, garantindo compatibilidade com o modelo nativo de exceções e permitindo a utilização do método virtual *what()* para a obtenção de mensagens descritivas. A partir desta, deriva a classe *InvalidDataException*, que assume o papel de exceção genérica do sistema, funcionando como elemento central na validação de dados introduzidos pelo utilizador. Esta classe integra um atributo destinado a armazenar a mensagem de erro e redefine o método *what()* para assegurar a sua correta apresentação. Por sua vez, a estrutura inclui especializações desta exceção geral, como *InvalidDateException*, utilizada em situações de inconsistência temporal, por exemplo datas inválidas, e *InvalidTypeException*, aplicada quando o tipo de dados inserido não corresponde ao esperado. Esta organização hierárquica permite uma deteção precisa e um tratamento consistente de diferentes tipos de erro, contribuindo para a robustez e fiabilidade global da aplicação.

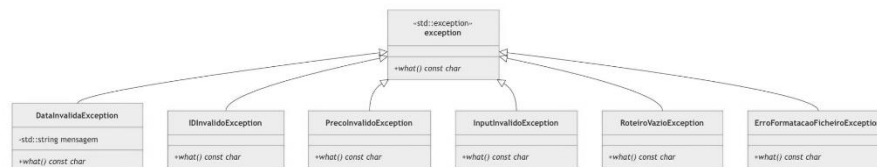


Fig. 6. Diagrama de exceções.

6 Diagrama de views

O Diagrama de Classes de Views descreve a arquitetura da interface do SGPV, detalhando como o sistema organiza a apresentação de dados e a captura de inputs. Diferente das classes lógicas, as classes de "View" têm a responsabilidade exclusiva de formatar a consola, exibir menus e validar a interação imediata do utilizador antes de

comunicar com o núcleo do sistema (Model). A estrutura está organizada em módulos de responsabilidade, visíveis através das associações no diagrama.

6.1 Consulta e gestão do catálogo

A Figura 7 apresenta a organização da camada de interface de utilizador do sistema, evidenciando a divisão por módulos funcionais que asseguram uma interação clara e estruturada com o utilizador. No âmbito da navegação e exploração do catálogo, destacam-se componentes responsáveis pela gestão do fluxo principal e pela apresentação organizada dos dados, permitindo uma visualização clara das opções disponíveis. No que diz respeito ao acesso e gestão de identidade, existem elementos dedicados à autenticação e registo, garantindo que o utilizador recebe feedback imediato e compreensível relativamente ao sucesso ou falha das operações. O núcleo operacional da interface centra-se no planeamento e edição de viagens, integrando funcionalidades que permitem definir parâmetros iniciais, alterar informações de viagens já existentes e visualizar a componente temporal através de representações cronológicas articuladas com o módulo de gestão de datas. Por fim, a fase de finalização é suportada por componentes que traduzem os cálculos financeiros em informação acessível, incluindo resumos de custos, aplicação de IVA e confirmação de pagamentos, sendo complementada por um sistema de registo que assegura a comunicação de mensagens, erros e estados do sistema.

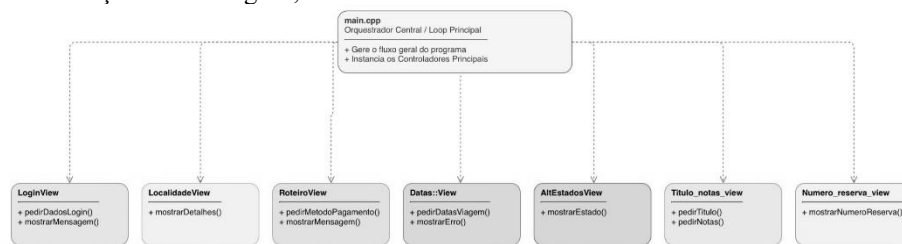


Fig. 7. Diagrama de views.

7 Diagrama de controladores

O Diagrama de Classes de Controladores representa o "cérebro" do SGPV. Esta camada é responsável por mediar a comunicação entre as Interfaces (Views) e a Lógica de Negócio (Model). Os controladores interpretam os comandos do utilizador, decidem quais os dados a carregar e determinam qual a próxima vista a ser apresentada, garantindo um fluxo de navegação lógico e coeso.

7.1 Arquitetura de controlo

A Figura 8 ilustra a organização da camada de controlo do sistema, evidenciando a forma como os diferentes controladores se articulam para assegurar o funcionamento global da aplicação. No topo desta estrutura destaca-se a orquestração central, assegurada pelo *MainMenuController*, que atua como ponto principal de decisão, recebendo as opções do utilizador e delegando o controlo para os restantes componentes especializados, como os responsáveis pela autenticação, catálogo ou planeamento. Paralelamente, o *LoggerController* desempenha um papel transversal e crítico, sendo responsável pelo registo de eventos, gestão de erros e comunicação de feedback, tal como evidenciado pelas múltiplas ligações com os restantes controladores. No que diz respeito à gestão dos fluxos operacionais, os módulos de autenticação asseguram a ligação entre o acesso do utilizador e a persistência de sessões, enquanto o planeamento de viagens é coordenado por um controlador específico que interage com componentes dedicados à gestão de viajantes, agendas e calendários, garantindo a correta definição de todos os parâmetros logísticos e temporais. Adicionalmente, o sistema integra controladores destinados à gestão de histórico e edição, permitindo recuperar viagens previamente guardadas e efetuar alterações de forma controlada. Por fim, a componente associada ao catálogo e ao processo de venda é assegurada por controladores responsáveis pela filtragem e preparação de dados para apresentação, bem como por um conjunto de módulos que tratam da organização das escolhas do utilizador e da finalização do processo, incluindo o cálculo financeiro e a emissão de informação associada ao pagamento.

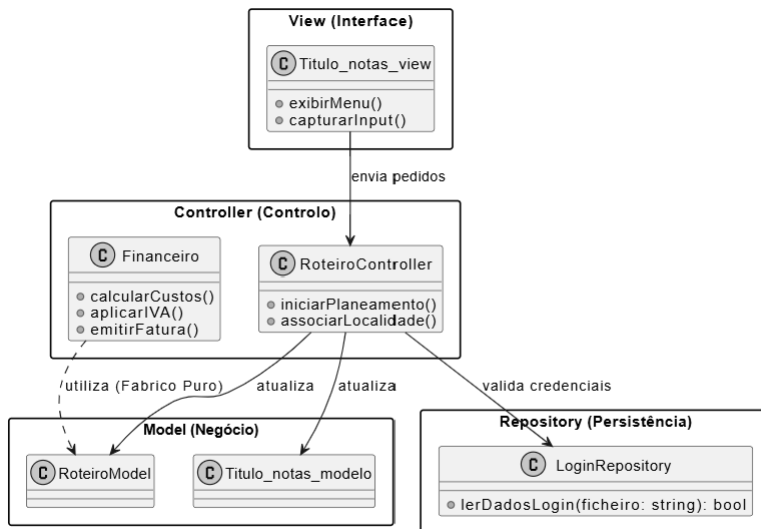


Fig. 8. Diagrama de controladores.

8 Diagrama de sequencia (SD)

Os diagramas de sequência abaixo detalham a interação dinâmica entre os objetos do sistema, demonstrando como as mensagens são trocadas para realizar uma funcionalidade específica. Eles são cruciais para validar se a lógica de controlo (Controllers) está devidamente separada da lógica de dados (Models)

- SD01: Finalizar Operação com Persistência Cumulativa:

Este diagrama de sequência ilustra a fase de encerramento e salvaguarda do sistema SGPV, representando o momento em que a informação processada durante a sessão é imortalizada no armazenamento secundário. O objetivo é demonstrar o mecanismo de Persistência de Dados, garantindo que o planeamento realizado pelo utilizador não se perca após o fecho da aplicação. A sequência detalha a orquestração entre o controlador de sistema e os fluxos de escrita de ficheiros (*File Streams*).

- SD02: Indexar Atividades e Horários Cronológicos

Este caso de uso detalha o processo de organização temporal de uma viagem, onde o utilizador define a agenda específica para cada destino selecionado. O objetivo central é demonstrar como o sistema gere a inserção de dados descritivos e horários, garantindo que estes fiquem vinculados à instância correta da viagem. A sequência foca-se na receção de inputs textuais e na sua conversão em elementos estruturados dentro da lista dinâmica de eventos da viagem.

- SD03: Instanciar Período Temporal com Captura de Exceções Logísticas

Este diagrama de sequência foca-se na robustez do sistema durante a definição do intervalo de tempo de uma viagem. O objetivo principal é demonstrar o funcionamento do mecanismo de Captura de Exceções Logísticas, garantindo que o software reaja de forma inteligente a erros de input antes que estes comprometam a lógica de negócio. O fluxo destaca o papel do controlador como um validador ativo, impedindo a progressão do programa caso os dados temporais sejam incoerentes.

- SD04: Executar Processamento Financeiro e Faturação (Checkout)

Este diagrama de sequência descreve o módulo financeiro do SGPV, representando o momento em que o plano de viagem é convertido num orçamento detalhado. O objetivo é demonstrar a integridade do cálculo de custos, garantindo que o sistema aplica as taxas fiscais corretas de forma automatizada. A sequência ilustra a transição da escolha

subjetiva de destinos para a objetividade financeira, culminando na geração de uma fatura para o utilizador.

- SD05: Adicionar Elemento ao Roteiro Polimórfico (Carrinho)

Este diagrama de sequência detalha a funcionalidade central de personalização do itinerário, permitindo que o utilizador construa a sua viagem de forma modular. O conceito de "Polimorfismo" é aqui fundamental, pois permite que o sistema trate diferentes tipos de destinos (Cidades, Monumentos ou Atividades) de forma uniforme. O fluxo demonstra como o controlador gere a inserção de objetos dinâmicos numa lista de escolhas, mantendo a integridade da estrutura do roteiro

- SD06: Revisão Geral e Edição Polimórfica de Perfis

Este caso de uso descreve a funcionalidade de gestão e auditoria de perfis dentro do sistema. O objetivo é permitir que o Organizador reveja e modifique os dados dos viajantes associados ao plano, utilizando o conceito de Edição Polimórfica. A sequência foca-se na capacidade do sistema de aceder a diferentes tipos de perfis através de uma interface comum, garantindo que as atualizações de dados (como nomes, idades ou preferências) sejam refletidas instantaneamente na memória da aplicação.

- SD07: Povoar e Associar Lista Dinâmica de Viajantes

Este caso de uso descreve o processo de definição do grupo de viagem, permitindo que o Organizador registre e vincule múltiplos participantes ao plano de viagem. O foco técnico aqui é a gestão de coleções dinâmicas, onde o sistema deve ser capaz de crescer conforme o número de viajantes introduzidos. A sequência ilustra a transição de dados individuais para uma estrutura de lista agregada, garantindo que cada perfil de viajante esteja corretamente associado ao nó principal do Organizador.

- SD08: Autenticação de Credenciais (Login de Conta)

Este diagrama de sequência ilustra a "porta de entrada" do sistema, detalhando o protocolo de segurança para a autenticação de utilizadores. O objetivo é demonstrar como o sistema valida a identidade do utilizador antes de conceder acesso aos planos de viagem privados. A sequência foca-se no processo de verificação cruzada entre os dados inseridos na interface e os registos armazenados na base de dados persistente do sistema.

- SD09: Registar Organizador com Controlo de Menoridade Legal

Este diagrama de sequência descreve o fluxo de criação de conta para o perfil principal do sistema: o Organizador. A particularidade técnica deste caso de uso reside no

mecanismo de Validação de Maioridade, uma regra de negócio crítica que impede que indivíduos sem capacidade jurídica plena assumam a responsabilidade por planos de viagem. O sistema atua como um inspetor de dados, validando a elegibilidade do utilizador antes de permitir a persistência do perfil na base de dados.

- SD10: Carregar Base de Dados do Catálogo (Inicialização do Sistema)

Este diagrama de sequência ilustra o procedimento de bootstrapping ou inicialização do SGPV, focando-se na transição de dados do armazenamento estático para o ambiente de execução dinâmico. O objetivo é demonstrar como o sistema popula o catálogo de destinos (Países, Cidades e Atividades) logo após o arranque, garantindo que o utilizador tenha informação disponível para consulta imediata. O fluxo destaca a interação entre o motor de arranque do sistema e a camada de persistência de ficheiros.

- SD11: Selecionar País com Validação de Tipo de Dados (Input Shield)

Este diagrama de sequência detalha o processo de interação segura entre o utilizador e o catálogo do sistema. O foco principal é o mecanismo de Input Shielding, que atua como uma barreira de proteção na camada da View. O objetivo é garantir que apenas comandos válidos cheguem à lógica de controlo, prevenindo falhas comuns de conversão de dados. A sequência ilustra como o sistema valida a natureza do input antes de tentar processar a seleção de um país.

- SD12: Consultar Catálogo Paginado de Países

Este diagrama de sequência ilustra a funcionalidade de exploração do catálogo, focando-se na eficiência da apresentação de dados através do mecanismo de Paginação Dinâmica. O objetivo é demonstrar como o sistema recupera coleções de objetos da memória e os organiza em "páginas" numeradas, facilitando a navegação do utilizador. A sequência destaca a separação entre o armazenamento total de dados e a renderização parcial no ecrã, garantindo uma interface limpa e performante.

- SD13: Alterar Designação do Plano com Validação de Extensão (Título)

Este diagrama de sequência ilustra o processo de edição de metadados de uma viagem, especificamente a alteração do seu título. O foco técnico reside na Integridade de Atributos, onde o sistema valida o comprimento da string inserida antes de permitir a alteração na memória RAM. O fluxo utiliza um mecanismo de captura de exceções para garantir que o título respeita os limites de usabilidade definidos no código, evitando títulos vazios ou excessivamente longos que possam comprometer a interface.

- SD14: Consulta de Identificador Único de Reserva

Este diagrama de sequência descreve uma funcionalidade de consulta direta e essencial para o utilizador: a recuperação do número de reserva associado ao seu plano de viagem. O objetivo deste caso de uso é demonstrar a eficiência do sistema no acesso a atributos específicos de uma instância, garantindo que o utilizador tenha sempre acesso rápido à sua referência de confirmação. O fluxo é desenhado de forma linear para minimizar a latência entre o pedido do utilizador e a resposta do sistema.

- SD15: Alteração do Método de Pagamento com Tratamento de Exceções

Este diagrama de sequência detalha a funcionalidade de gestão financeira que permite ao utilizador definir ou alterar a forma como tenciona liquidar o custo da viagem. O objetivo central é demonstrar a Robustez do Sistema de Input, garantindo que apenas opções de pagamento suportadas pela lógica de negócio sejam aceites. O fluxo destaca a proteção contra seleções inválidas através do lançamento de exceções, assegurando que o estado financeiro da viagem permaneça sempre consistente.

- SD16: Alteração da Data e Hora de Reserva com Validação

Este diagrama de sequência descreve o protocolo de modificação temporal de uma reserva existente. Diferente de uma simples inserção de texto, este caso de uso exige uma Validação de Atributos Múltiplos, onde cada componente da data (ano, mês, dia) e da hora (horas, minutos) é verificado individualmente. O sistema utiliza uma estrutura de diálogo interativo que impede a progressão do fluxo caso qualquer um dos valores inseridos seja logicamente impossível, garantindo a integridade cronológica do SGPV.

- SD17: Gerar Relatório Consolidado de Viagem

Este diagrama de sequência descreve o processo de saída de dados (Output) final do sistema. O objetivo é demonstrar a capacidade de Agregação de Dados, onde diferentes objetos e atributos são chamados em cascata para formar um resumo legível e completo da viagem. O fluxo ilustra a cooperação entre a classe principal e as suas classes auxiliares (como a Data) para garantir que nenhuma informação relevante seja omitida na visualização final.

- SD18: Alteração de Notas da Viagem com Validação de Extensão

Este diagrama de sequência descreve a funcionalidade que permite ao utilizador anexar ou modificar observações personalizadas ao seu plano de viagem (ex: "Levar casaco quente" ou "Confirmar transfer"). Tal como na edição do título, este processo não é apenas uma inserção direta de texto; ele inclui uma camada de Validação de Integridade para garantir que o campo de notas não seja utilizado de forma indevida (ex: textos

vazios ou demasiado extensos que possam quebrar a formatação do relatório final). O sistema utiliza um bloco de proteção para validar o input antes da gravação em memória

- SD19: Criar Roteiro e Adicionar Horários (Processamento Cronológico)

Este diagrama de sequência detalha o núcleo logístico do sistema: o momento em que o utilizador define a moldura temporal da sua viagem. O foco aqui é o Cálculo Automático de Duração e a Validação Interna de Calendário. Diferente de outros fluxos, este demonstra uma interação complexa entre quatro componentes distintos, garantindo que as datas de chegada e saída são logicamente possíveis e que o roteiro é construído sobre uma base temporal sólida. O sistema não se limita a guardar números, mas processa-os para gerar informação útil, como a contagem total de dias.

9 GRASP e SOLID

Para garantir uma arquitetura robusta, escalável e de fácil manutenção, o desenvolvimento do sistema da nossa Agência de Viagens foi alicerçado nos princípios SOLID e nos padrões GRASP. Enquanto o SOLID assegura que as classes são modulares e fáceis de estender sem quebrar o sistema, o GRASP orienta a correta atribuição de responsabilidades a cada objeto. Neste capítulo, demonstramos como aplicamos estes conceitos na prática ao longo do nosso código C++, garantindo alta coesão, baixo acoplamento e uma clara separação de responsabilidades.

9.1 SOLID

Os princípios SOLID focam-se na organização lógica e na estrutura das classes, minimizando o acoplamento rígido e garantindo que o sistema seja modular e fácil de estender à medida que novos requisitos surgem na programação da Agência de Viagens.

9.1.1 S - Single Responsibility Principle (Princípio da Responsabilidade Única)

Uma classe deve ter apenas um motivo para mudar, ou seja, deve ter apenas uma responsabilidade ou propósito no sistema.

Exemplo: Classe View.

```
class View {  
  
public:  
  
    DadosBrutos pedirDatasViagem();  
  
    void mostrarErro(const std::string& erro);
```

```

        void mostrarSucesso(const std::string& ch, const
std::string& sa, int dias, const
std::vector<std::string>& horarios);

};

```

Esta classe não sabe calcular datas, não guarda listas nem valida se um dia existe. A sua única responsabilidade é a interação com o utilizador (pedir dados via `std::cin` e imprimir resultados no `std::cout`). Qualquer alteração na interface (ex: mudar para interface gráfica) afeta apenas esta classe.

9.1.2 O - Open/Closed Principle (Princípio do Aberto/Fechado)

As classes devem estar abertas para extensão, mas fechadas para modificação.

Exemplo: Hierarquia da classe Pessoa ou LocalidadeModel.

```

class Pessoa { ... };

class Viajante : public Pessoa { ... };

class Organizador : public Pessoa { ... };

```

Se futuramente a agência necessitar de um novo tipo de interveniente, como um GuiaTuristico, não precisamos de alterar o código já testado da classe Pessoa. Basta criarmos uma nova classe `class GuiaTuristico : public Pessoa`. O sistema está aberto a extensões, mantendo o código base intocado.

9.1.3 L - Liskov Substitution Principle (Princípio da Substituição de Liskov)

As classes filhas devem poder substituir as classes base sem quebrar a semântica e o funcionamento do programa.

Exemplo: A utilização de polimorfismo com LocalidadeModel e CidadeModel na classe Financeiro.

```

// No ficheiro Financeiro.cpp

double Financeiro::calcularTotalBase(const
std::vector<LocalidadeModel*>& listaParaContas) {

    double somaTotal = 0.0;

    for (LocalidadeModel* ponteiroLocalidade :
listaParaContas) {

        if (ponteiroLocalidade != nullptr) {

            // ponteiroLocalidade pode apontar para uma
CidadeModel ou AtividadeModel

```

```

        somaTotal += ponteiroLocalidade->obterPreco();
    }

    return somaTotal;
}

```

Sempre que o sistema espera um objeto do tipo genérico `LocalidadeModel*` (como no vetor iterado na função acima), podemos passar uma instância concreta de `CidadeModel`. Quando o sistema invoca `ponteiroLocalidade->obterPreco()`, a classe derivada devolve o seu preço específico corretamente. A classe base é estendida sem que o seu comportamento esperado (o contrato) seja corrompido, provando que a classe filha se comporta perfeitamente como o seu tipo genérico de base.

9.1.4 I - Interface Segregation Principle (Princípio da Segregação de Interfaces)

Uma classe não deve ser forçada a implementar interfaces/métodos que não vai utilizar. Interfaces devem ser específicas e não "gordas" (fat interfaces).

Exemplo: Separação estrita entre a interface de apresentação e a interface de dados.

```

// 1. Interface de Dados: LocalidadeModel.h (Apenas
carrega dados, não usa <iostream>)

class LocalidadeModel {

public:

    virtual double obterPreco() const;

    virtual void carregarDeLinha(std::string linha) =
0; // Método obrigatório focado apenas nos dados

};

// 2. Interface de Apresentação: LocalidadeView.h (Lida
exclusivamente com Output no ecrã)

class LocalidadeView {

public:

    void mostrarDetalhes(const LocalidadeModel*
localidade) const;

```

```

        void mostrarErroFicheiro() const;

};

```

Em vez de forçarmos os modelos (como CidadeModel ou AtividadeModel) a terem métodos virtuais intrusivos que não lhes interessam (como virtual void mostrarDetalhesNoEcrã() = 0;), nós segregamos essas responsabilidades. O modelo abstrato de dados exige apenas que as suas filhas implementem o que é da sua responsabilidade a nível de memória/lógica (carregarDeLinha). Por sua vez, os métodos de visualização estão segregados para a LocalidadeView. Desta forma, nenhuma classe no projeto é obrigada a implementar ou herdar métodos que não vai efetivamente utilizar.

9.1.5 D - Dependency Inversion Principle (Princípio da Inversão de Dependência)

Módulos de alto nível não devem depender de módulos de baixo nível. Ambos devem depender de abstrações.

Exemplo:

```

double calcularTotal(const
std::vector<LocalidadeModel*>& lista) {

    double total = 0;

    for (auto const& item : lista) {

        total += item->obterPreco();

    }

    return total;

}

```

O módulo que faz os cálculos globais (alto nível) não depende diretamente das classes concretas como CidadeModel ou AtividadeModel (baixo nível). Depende apenas da abstração LocalidadeModel*. Isto reduz o acoplamento e flexibiliza o código.

9.2 GRASP

Os padrões GRASP (*General Responsibility Assignment Software Patterns*) funcionam como um guia prático para a correta atribuição de responsabilidades a cada objeto (decidindo quem deve criar, quem deve validar ou quem deve controlar o fluxo), promovendo a estabilidade e a coesão do software.

9.2.1 Information Expert (Especialista na Informação)

Atribuir a responsabilidade à classe que possui a informação necessária para a cumprir.

Exemplo:

```

bool Data::validarData(int d, int m, int a) const {

    if (a < 2024) return false;

    if (m < 1 || m > 12) return false;

    // validações de dias e anos bissextos...

    return true;

}

```

Quem deve validar quantos dias um mês tem é a classe Data. Sendo ela a detentora da informação (variáveis de tempo), é a "especialista" lógica na sua própria consistência.

9.2.2 Creator (Criador)

Quem deve ser responsável por criar uma nova instância de uma classe é aquele que a agrega, contém ou utiliza diretamente.

Exemplo: O método obterDataValidada() dentro da classe Controller.

```

// No ficheiro Controller.cpp

Data Controller::obterDataValidada() {

    while (true) {

        // 1. Recebe os dados de inicialização através
        da View

        DadosBrutos dados =
        interfaceUsuario.pedirDatasViagem();

        try {

            // 2. O Controller atua como "Creator"
            instanciando o objeto 'viagem'

            Data viagem(dados.diaC, dados.mesC,
            dados.anoC, dados.diaS, dados.mesS, dados.anoS);

            while
            (interfaceUsuario.perguntarMaisHorarios()) {

                std::string hor =
                interfaceUsuario.pedirHorario();

                viagem.addHorario(hor);
            }
        }
    }
}

```

```

    }

    // ...

    return viagem;

} catch (const DataInvalidaException& e) {

    interfaceUsuario.mostrarErro(e.what());

}

}

}

```

A classe Controller recebe os "dados brutos" do utilizador através da View e é a entidade lógica responsável por invocar o construtor: `Data viagem(dados.diaC, ...)`. Uma vez que o Controlador gere o fluxo de informação, compete-lhe instanciar o modelo de domínio principal.

9.2.3 Polymorphism (Polimorfismo)

Quando o comportamento varia de acordo com o tipo, deve-se atribuir a responsabilidade usando operações polimórficas.

Exemplo:

```

double AtividadeModel::obterPreco() const override {

    return precoTicket;

}

double CidadeModel::obterPreco() const override {

    return precoCidade;

}

```

Evitam-se *ifs* ou *switches* extensos para verificar qual é o tipo de localidade na hora de somar preços. O sistema confia no método virtual `obterPreco()`, acionando dinamicamente a versão correta da classe derivada em tempo de execução.

9.2.4 Pure Fabrication (Fabricação Pura)

Criar uma classe artificial (que não representa um conceito direto do domínio/mundo real) para manter a alta coesão e o baixo acoplamento.

Exemplo: A classe View ou a classe Controller.

```

// Ficheiro View.h

```

```

class View {

public:

    DadosBrutos pedirDatasViagem();

    void mostrarErro(const std::string& erro);

    void mostrarSucesso(const std::string& ch, const
std::string& sa, int dias, const
std::vector<std::string>& horarios);

    bool perguntarMaisHorarios();

    std::string pedirHorario();

};

```

No mundo real de uma agência de viagens, não existe o conceito tangível de um "Controlador" ou de uma "View". Estas classes foram "fabricadas" puramente por conveniência de software, evitando que objetos de domínio como a Data ou Pessoa fossem poluídos com lógica de interações do terminal.

9.2.5 Controller (Controlador)

Atribuir a responsabilidade de lidar com eventos do sistema a uma classe que represente o cenário global ou o caso de uso.

Exemplo: A classe Controller.

```

// Ficheiro Controller.cpp

Data Controller::obterDataValidada() {

    while (true) {

        // Delega o input à View

        DadosBrutos dados =
interfaceUsuario.pedirDatasViagem();

        try {

            // Cria o modelo

            Data viagem(dados.diaC, dados.mesC,
dados.anoC, dados.diaS, dados.mesS, dados.anoS);

```



```

        // Orquestra a adição de horários

        while
(interfaceUsuario.perguntarMaisHorarios()) {

            std::string hor =
interfaceUsuario.pedirHorario();

            viagem.addHorario(hor);

        }

        // ...

        return viagem;

    } catch (const DataInvalidaException& e) {

        interfaceUsuario.mostrarErro(e.what());

    }

}
}

```

Controller::obterDataValidada() atua como um maestro. É ele que orquestra a chamada à View para pedir informações e as injeta nas classes de negócio/modelos (como a classe Data), decidindo o rumo do programa sem implementar a lógica de dados internamente.

9.2.6 High Cohesion (Alta Coesão)

As responsabilidades de uma classe devem estar fortemente relacionadas e focadas num único propósito.

Exemplo: A classe Data.

```

// Ficheiro Data.cpp

bool Data::validarData(int d, int m, int a) const {

    if (a < 2024) return false;

    if (m < 1 || m > 12) return false;

    // ... validações de anos bissextos

    return true;
}

```

```

}

int Data::dataParaDias(int d, int m, int a) const {

    if (m < 3) { m += 12; a--; }

    return 365 * a + a / 4 - a / 100 + a / 400 + (153 *
m + 2) / 5 + d;

}

void Data::atualizarNumeroDias() {

    int diasC = dataParaDias(chegada.dia, chegada.mes,
chegada.ano);

    int diasS = dataParaDias(saida.dia, saida.mes,
saida.ano);

    numeroDias = diasS - diasC;

}

```

A classe Data tem altíssima coesão porque todos os seus métodos (validarData, dataParaDias, atualizarNumeroDias) estão estritamente focados num único tema: gestão e verificação de cronologias. Ela não tenta formatar texto para a consola nem gravar em base de dados.

9.2.7 Low Coupling (Baixo Acoplamento)

Minimizar as dependências entre classes para que a alteração numa não cause um efeito dominó noutras.

Exemplo: Controller.h / Controller.cpp

```

// Ficheiro Controller.h

#include "View.h"

#include "data.h"

class Controller {

private:

    View interfaceUsuario;

```

```

public:

    Data obterDataValidada();

};

```

Controller delega todo o Output/Input à interface View e o tratamento dos dias à Data. Se for necessário alterar como a validação dos anos bissextos funciona internamente no Data.cpp, o código do Controller não precisa de ser modificado, demonstrando um grau de dependência mínimo e estruturado.

10. Implementação e Boas Práticas de POO

A transição dos modelos arquiteturais teóricos para código C++ funcional no projeto da nossa Agência de Viagens exigiu a aplicação rigorosa de boas práticas de Programação Orientada a Objetos (POO). Com o objetivo de garantir um código limpo, seguro contra falhas de memória e que facilite o trabalho colaborativo da equipa, estabelecemos diretrizes claras ao longo de todo o desenvolvimento. De seguida, detalhamos as principais regras adotadas para assegurar a qualidade e a manutenibilidade do sistema.

10.1 Regras Estruturais:

O código foi estritamente dividido em dois tipos de ficheiros, ficheiros do tipo Headers (.h), ficheiros dedicados exclusivamente à declaração das classes e protótipos de métodos. Ficheiros do tipo Sources (.cpp), ficheiros que contêm a implementação lógica.

Exemplo: (Separação da classe CidadeModel):

Header (CidadeModel.h):

```

class CidadeModel : public LocalidadeModel {

protected:

    int paisAssociado;

    double precoCidade;

public:

    CidadeModel();

    int obterIdPais() const;

};

```

Source (CidadeModel.cpp):

```
#include "CidadeModel.h"

int CidadeModel::obterIdPais() const {

    return paisAssociado;

}
```

A declaração da interface da classe e das respetivas assinaturas é realizada no ficheiro .h, enquanto a implementação detalhada da lógica algorítmica se encontra no ficheiro .cpp.

10.2 Encapsulamento de Dados

Para proteger a integridade (Encapsulamento), atributos críticos foram declarados como private ou protected.

Exemplo: Data e Modelos.

```
// Exemplo 1: Encapsulamento estrito na classe Data

class Data {

private:

    // Atributos inacessíveis diretamente pelo exterior

    Calendario chegada;

    Calendario saida;

    int numeroDias;

public:

    Data(int diaC, int mesC, int anoC, int diaS, int
mesS, int anoS);

    // Comunicação com o exterior feita via getters
'const' (read-only)

    int getNumeroDias() const {

        return numeroDias;

    }

}
```

```

        Calendario getChegada() const {

            return chegada;

        }

};

// Exemplo 2: Uso de 'protected' na classe CidadeModel

class CidadeModel : public LocalidadeModel {

protected:

    // Atributos protegidos, acessíveis apenas pela
    // classe e as suas filhas

    int paisAssociado;

    double precoCidade;

public:

    CidadeModel();

    // Getter read-only que não adultera o estado do
    // objeto

    double obterPreco() const override {

        return precoCidade;

    }

};

```

Os atributos como as datas de chegada e saída ou o `precoCidade` não podem ser adulterados de forma externa (por exemplo, a partir do `main` ou da classe `Controller`). A comunicação com o exterior faz-se estritamente através de métodos de leitura (*getters*) que utilizam a cláusula `const` (ex: `int getNumeroDias() const`). Isto garante que o estado do objeto se mantém inviolável e estritamente *read-only* durante as operações de acesso e visualização.

10.3 Convenções de Nomenclatura

UpperCamelCase: Para Classes e Exceções Customizadas. (Ex: CidadeModel, DataInvalidaException).

lowerCamelCase: Para atributos e métodos. (Ex: obterPreco(), perguntarMaisHorarios(), numeroDias).

11. Gestão de erros

O tratamento de erros foi estruturado com base num sistema robusto de lançamento e captura de exceções personalizadas (herdando da biblioteca <stdexcept>), sinalizando violações de regras de negócio.

Exemplo:

```
class DataInvalidaException : public std::exception {  
  
private:  
  
    std::string mensagem;  
  
public:  
  
    DataInvalidaException(std::string msg) :  
    mensagem(msg) {}  
  
    const char* what() const noexcept override { return  
    mensagem.c_str(); }  
  
};
```

Na camada intermédia, como no construtor de Data, a exceção é lançada (throw DataInvalidaException) quando detetada uma quebra do domínio (ex: mês inválido ou datas negativas).

Já na camada de controlo, a interação do utilizador é blindada com blocos try-catch, impedindo o encerramento abrupto do programa. No Controller::obterDataValidada(), o sistema apanha (catch) a exceção e envia o objeto de erro para a View, permitindo ao utilizador reintroduzir os dados em segurança num ciclo infinito (while(true)) até as condições de sucesso serem alcançadas.

Exemplo: Controller.cpp:

```
try {  
  
    Data viagem(dados.diaC, dados.mesC, dados.anoC,  
    dados.diaS, dados.mesS, dados.anoS);  
  
    // ...
```

```

        return viagem;

    } catch (const DataInvalidaException& e) {

        interfaceUsuario.mostrarErro(e.what());

    }

```

12. Testes

Teste: Construtor por Omissão (Default)

Este teste garante que, quando criamos uma Pessoa sem passar nenhum dado, os valores iniciais estão corretos (em C++, inteiros não inicializados ganham valores "lixo", por isso testamos se a idade começou mesmo a zero).

```

TEST(PessoaTest,
ConstrutorPorOmissao_InicializaIdadeAZero) {

    // Arrange & Act: Criamos o objeto sem passar
    argumentos

    Pessoa p;

    // Assert: Verificamos se o construtor colocou a
    idade a 0 e o nome vazio

    EXPECT_EQ(p.obterIdade(), 0);

    EXPECT_TRUE(p.obterNome().empty());

}

```

Teste: Construtor Parametrizado Este teste verifica se, ao fornecermos todos os dados no momento da criação do objeto, eles são guardados nas variáveis corretas da classe.

```

TEST(PessoaTest,
ConstrutorParametrizado_AtribuiDadosCorretamente) {

    // Arrange: Preparamos os dados que vamos usar para
    criar a pessoa

    std::string nome = "Joao Silva";

    int idade = 30;

    std::string email = "joao@email.com";

```

```
std::string bi = "12345678";
```

```
std::string validade = "12/2028";
```

Teste: Getters e Setters (Exemplo com Nome) Testa a capacidade de alterar um valor de um objeto que já existe e, de seguida, recuperá-lo com sucesso. (A lógica é idêntica para Email, BI, Validade e Idade).

```
TEST(PessoaTest, ColocarEObterNome_Sucesso) {  
  
    // Arrange: Criamos a pessoa e definimos o valor  
    esperado  
  
    Pessoa p;  
  
    std::string nomeEsperado = "Ana";  
  
    // Act: Usamos a função de set (colocar)  
  
    p.colocarNome(nomeEsperado);  
  
    // Assert: Usamos a função de get (obter) e  
    comparamos  
  
    EXPECT_EQ(p.obterNome(), nomeEsperado);  
}  
  
// Act: Criamos o objeto passando os dados  
  
Pessoa p(nome, idade, email, bi, validade);  
  
// Assert: Verificamos se cada getter devolve  
exatamente o que inserimos  
  
EXPECT_EQ(p.obterNome(), nome);  
  
EXPECT_EQ(p.obterIdade(), idade);  
  
EXPECT_EQ(p.obterEmail(), email);  
  
EXPECT_EQ(p.obterBI(), bi);  
  
EXPECT_EQ(p.obterValidade(), validade);  
}
```


Teste: Operador de Igualdade (Caso o tenhas implementado) Se adicionares um operador para comparar Pessoas através do BI, este teste valida se a lógica do operador funciona, ou seja, se duas instâncias diferentes mas com o mesmo BI são consideradas "a mesma pessoa".

```
TEST(PessoaTest, OperadorIgualdade_MesmoBI_RetornaTrue)
{
    // Arrange: Duas instâncias diferentes de Pessoa

    Pessoa p1, p2;

    p1.colocarBI("111222"); // Mesmo BI
    p2.colocarBI("111222"); // Mesmo BI

    // Act: Acionamos o operador ==

    bool saoIguais = (p1 == p2);

    // Assert: O resultado tem de ser verdadeiro

    EXPECT_TRUE(saoIguais);
}
```

Teste: Construtor do Viajante (Herança) Como o Viajante herda de Pessoa, este teste certifica-se de que a mecânica de herança do C++ está a funcionar e que as propriedades da classe base (Pessoa) estão a ser inicializadas corretamente pelo construtor do Viajante.

```
TEST(ViajanteTest,
ConstrutorDefault_InstanciaCorretamente) {
    // Arrange & Act: Criar um Viajante vazio

    Viajante v;

    // Assert: A idade deve estar a 0 (herdado de
    Pessoa)

    EXPECT_EQ(v.obterIdade(), 0);
}
```

Teste: Adicionar Viajantes ao Organizador Este teste valida se o método adicionarViajante insere efetivamente o objeto no vetor (std::vector<Viajante>) e se a contagem global é atualizada.

```

TEST(OrganizadorTest,
AdicionarViajante_AumentaO_NumeroDeViajantes) {

    // Arrange: Preparar o organizador e um viajante

    Organizador org;

    Viajante v;

    v.colocarNome("Carlos");

    // Act: Adicionar à lista

    org.adicionarViajante(v);

    // Assert: Confirmar que a lista tem tamanho 1 e
    que o nome bate certo

    EXPECT_EQ(org.obterNumeroViajantes(), 1);

    EXPECT_EQ(org.obterViajantes()[0].obterNome(),
"Carlos");

}

```

Teste: Limpar a lista de Viajantes Verifica o método limparViajantes(). A intenção é garantir que, depois de chamada a função, não sobra nenhum viajante na lista (útil para quando o utilizador se engana e quer recomeçar).

```

TEST(OrganizadorTest, LimparViajantes_EsvaziaALista) {

    // Arrange: Inserir viajantes fictícios

    Organizador org;

    Viajante v1, v2;

    org.adicionarViajante(v1);

    org.adicionarViajante(v2);

    // Act: Limpar

    org.limparViajantes();

    // Assert: Confirmar contagem a zero e lista vazia

    EXPECT_EQ(org.obterNumeroViajantes(), 0);

```

```
        EXPECT_TRUE(org.obterViajantes().empty());  
    }  
}
```

Teste: Substituir Viajante (Sucesso) Este teste valida uma regra de negócio importante: só podemos substituir um viajante se o índice fornecido for válido.

```
// Caminho de Sucesso
```

```
TEST(OrganizadorTest,  
    SubstituirViajante_IndiceValido_AlteraO_Viajante) {  
    // Arrange: Ter um viajante na posição 0  
  
    Organizador org;  
  
    Viajante vAntigo;  
  
    vAntigo.colocarNome("Antigo");  
  
    org.adicionarViajante(vAntigo);  
  
    Viajante vNovo;  
  
    vNovo.colocarNome("Novo");  
  
    // Act: Tentar substituir na posição 0 (que existe)  
  
    bool resultado = org.substituirViajante(0, vNovo);  
  
    // Assert: Tem de retornar true e o nome na posição  
    // 0 tem de ter mudado  
  
    EXPECT_TRUE(resultado);  
  
    EXPECT_EQ(org.obterViajantes()[0].obterNome(),  
        "Novo");  
}
```

Teste: Substituir Viajante (Falha) Este teste valida uma regra de negócio importante: só podemos substituir um viajante se o índice fornecido for válido.

```

// Caminho de Falha

TEST(OrganizadorTest,
SubstituirViajante_IndiceInvalido_RetornaFalso) {

    // Arrange: Um organizador sem viajantes (lista vazia)

    Organizador org;

    Viajante vNovo;

    // Act: Tentar substituir na posição 0 (que NÃO existe)

    bool resultado = org.substituirViajante(0, vNovo);

    // Assert: Tem de retornar falso e não pode rebentar (crashar) o programa

    EXPECT_FALSE(resultado);

}

```

Teste: Construtor Parametrizado de Login Garante que a classe responsável por guardar os dados de acesso da sessão atribui corretamente o email, a password e o utilizador (Organizador) logado.

```

TEST(LoginTest,
ConstrutorEGetters_InicializaCorretamente) {

    // Arrange: Preparar o organizador

    Organizador org;

    org.colocarNome("Admin");

    // Act: Criar o login com dados concretos

    Login login("admin@empresa.pt", "pass123", org);

    // Assert: Validar se todos os dados ficaram guardados

    EXPECT_EQ(login.obterEmail(), "admin@empresa.pt");
}

```

```
        EXPECT_EQ(login.obterPassword(), "pass123");

        EXPECT_EQ(login.obterOrganizador().obterNome(),
"Admin");
    }
}
```

Teste: Organizador Maior de Idade (Válido)

```
TEST(AutenticacaoServiceTest,
OrganizadorTemIdadeMinima_Idade20_RetornaTrue) {

    // Arrange: Criar organizador com 20 anos

    AutenticacaoService service;

    Organizador org;

    org.colocarIdade(20);

    // Act: Verificar se tem idade

    bool pode = service.organizadorTemIdadeMinima(org);

    // Assert: Sim, deve retornar verdadeiro

    EXPECT_TRUE(pode);

}
```

Teste: Organizador Menor de Idade (Inválido)

```
TEST(AutenticacaoServiceTest,
OrganizadorTemIdadeMinima_Idade17_RetornaFalse) {

    // Arrange: Criar organizador com 17 anos

    AutenticacaoService service;

    Organizador org;

    org.colocarIdade(17);

    // Act: Verificar se tem idade

    bool pode = service.organizadorTemIdadeMinima(org);

    // Assert: Não, deve retornar falso

}
```

```
EXPECT_FALSE(pode);
```

```
}
```

Teste: Organizador no Limite (Exatamente 18)

```
TEST(AutenticacaoServiceTest,  
OrganizadorTemIdadeMinima_Idade18_RetornaTrue) {
```

```
    // Arrange: Criar organizador exatamente com a  
    idade limite
```

```
    AutenticacaoService service;
```

```
    Organizador org;
```

```
    org.colocarIdade(18);
```

```
    // Act: Verificar se tem idade
```

```
    bool pode = service.organizadorTemIdadeMinima(org);
```

```
    // Assert: Sim, a regra é "maior ou igual a 18",  
    logo tem de passar
```

```
    EXPECT_TRUE(pode);
```

```
}
```

Teste: Construtor Padrão (construtorPadrao)

O que faz: Quando o sistema cria uma localidade genérica sem qualquer informação preenchida, o modelo é obrigado a inicializar-se de forma limpa e segura. Este teste garante que, por padrão, o identificador (ID) comece em 0, o nome fique completamente vazio ("") e o preço base seja fixado em 0.0. Isto previne erros de memória no arranque do programa.

```
TEST(TesteLocalidadeModelo, construtorPadrao) {
```

```
    // Arrange & Act: Instanciação do objeto com o nome por  
    extenso
```

```
    TesteLocalidade localidadeInstanciada;
```

```
    // Assert: Verificação dos valores iniciais de fábrica
```

```
    EXPECT_EQ(localidadeInstanciada.obterID(), 0);
```

```
    EXPECT_EQ(localidadeInstanciada.obterNome(), "");
```

```
    EXPECT_DOUBLE_EQ(localidadeInstanciada.obterPreco(),  
0.0);
```

```
}
```

Teste: Construtor Parametrizado (construtorParametrizado)

O que faz: Para que o catálogo consiga registrar destinos reais, o sistema precisa de criar localidades passando dados específicos à partida. Este teste valida se, ao introduzir um ID (como 10) e um nome (como "Lisboa"), o modelo consegue receber, associar e guardar esses atributos corretamente nas suas variáveis internas, sem distorcer a informação.

```
TEST(TesteLocalidadeModelo, construtorParametrizado) {

    // Arrange: Definição das variáveis sem qualquer
    tipo de abreviação

    int identificadorEsperado = 10;

    std::string nomeEsperado = "Lisboa";

    // Act: Criação do objeto com os parâmetros
    definidos

    TesteLocalidade
    localidadeInstanciada(identificadorEsperado,
    nomeEsperado);

    // Assert: Validação dos dados retornados pelas
    funções do modelo

    EXPECT_EQ(localidadeInstanciada.obterID(),
    identificadorEsperado);

    EXPECT_EQ(localidadeInstanciada.obterNome(),
    nomeEsperado);

}
```

Teste: Preço Base (obterPrecoBase)

O que faz: Por se tratar de uma classe base conceptual, uma "localidade" pura não possui um custo financeiro próprio associado. Este teste certifica que o método polimórfico obterPreco() responde sempre com 0.0 EUR quando chamado a partir da classe base, assegurando que os preços reais só entrem em ação quando falamos especificamente de subclasses como Cidades ou Atividades.

```
TEST(TesteLocalidadeModelo, obterPrecoBase) {

    // Arrange: Configuração do cenário de teste

    TesteLocalidade localidadeInstanciada(1, "Porto");

    // Act: Execução do comportamento que queremos
    analisar
```

```

        double precoRetornado =
        localidadeInstanciada.obterPreco();

        // Assert: Garantia de que o preço base permanece
        em zero vírgula zero

        EXPECT_DOUBLE_EQ(precoRetornado, 0.0);

    }

```

Teste: Gestão de Atividades Internas (gerirAtividadesInternas)

O que faz: Para enriquecer a experiência do utilizador, o sistema permite associar uma lista de experiências e pontos de interesse textuais diretamente a um destino (por exemplo, adicionar "Visita às Caves do Vinho do Porto" ao Porto). Este teste verifica se a estrutura de dados interna consegue acumular estas atividades dinamicamente, contar o número total de registos e devolver os nomes exatos na ordem correta em que foram inseridos.

```

TEST(TesteLocalidadeModelo, gerirAtividadesInternas) {

    TesteLocalidade localidadeInstanciada(1, "Porto");

    // Act: Adicionar atividades diretamente na
    instância do modelo

    localidadeInstanciada.adicionarAtividade("Visita às
    Caves do Vinho do Porto");

    localidadeInstanciada.adicionarAtividade("Cruzeiro
    das Seis Pontes");

    // Assert: Verificar se foram guardadas e listadas
    corretamente

    const auto& listaRetornada =
    localidadeInstanciada.obterAtividades();

    EXPECT_EQ(listaRetornada.size(), 2);

    EXPECT_EQ(listaRetornada.at(0), "Visita às Caves do
    Vinho do Porto");

}

```

Teste: Construtor Padrão (construtorPadrao)

O que faz: Garante que, ao instanciar um objeto do tipo PaisModel sem passar nenhuma informação inicial, os atributos herdados da classe base são inicializados de forma correta e segura. O teste valida se o ID começa em 0, o nome fica vazio ("") e o preço

se mantém em 0.0. Isto serve para certificar que a herança com LocalidadeModel está a funcionar perfeitamente no momento da criação.

```
TEST(TestePaisModelo, construtorPadrao) {

    // Arrange & Act: Criação do objeto utilizando o
    construtor sem parâmetros

    PaisModel paisInstanciado;

    // Assert: Verificação se os atributos iniciam com
    os valores de fábrica

    EXPECT_EQ(paisInstanciado.obterID(), 0);

    EXPECT_EQ(paisInstanciado.obterNome(), "");

    EXPECT_DOUBLE_EQ(paisInstanciado.obterPreco(),
0.0);

}
```

6. Teste do Construtor Parametrizado (construtorParametrizado)

O que faz: Verifica se o construtor configurado para receber parâmetros consegue atribuir corretamente os valores de identidade de um país. O teste injeta o ID 351 e o nome "Portugal" e, de seguida, utiliza os métodos de leitura para confirmar que as variáveis internas guardaram exatamente esses dados, sem perdas ou corrupção de informação.

```
TEST(TestePaisModelo, construtorParametrizado) {

    // Arrange: Configuração dos elementos que serão
    injetados no construtor

    int identificadorEsperado = 351;

    std::string nomeEsperado = "Portugal";

    // Act: Criação do objeto com os parâmetros
    completamente por extenso

    PaisModel paisInstanciado(identificadorEsperado,
nomeEsperado);

    // Assert: Validação se os métodos de leitura
    retornam os dados corretos
```

```

        EXPECT_EQ(paisInstanciado.obterID(),
identificadorEsperado);

        EXPECT_EQ(paisInstanciado.obterNome(),
nomeEsperado);
    }

```

Teste de Leitura de Linha Válida (carregarDeLinhaValida)

O que faz: Valida o motor de parsing (processamento de texto) do modelo do país quando este recebe uma linha perfeitamente formatada a partir do ficheiro de texto (base de dados). O teste simula a entrada da linha "1;Portugal" e ativa o método carregarDeLinha(). O objetivo é garantir que a classe consegue detetar o separador ponto e vírgula (;), dividir a string com sucesso, converter o ID para inteiro e isolar o nome do país.

```

TEST(TestePaisModelo, carregarDeLinhaValida) {

    // Arrange: Preparação de uma linha de texto com o
formato correto de leitura

    PaisModel paisInstanciado;

    std::string linhaEntrada = "1;Portugal";

    paisInstanciado.carregarDeLinha(linhaEntrada);

    // Assert: Verificação se o identificador e o nome
foram extraídos com sucesso

    EXPECT_EQ(paisInstanciado.obterID(), 1);

    EXPECT_EQ(paisInstanciado.obterNome(), "Portugal");

}

```

Teste de Leitura de Linha Inválida (carregarDeLinhaInvalida)

O que faz: Avalia o comportamento de segurança e resiliência do modelo quando este é exposto a dados corrompidos ou mal formatados no ficheiro. O teste cria um país preenchido previamente (5;Franca) e tenta forçar a leitura de uma linha sem o separador correto ("TextoSemPontoEVirgula"). O teste garante que o sistema aborta a operação de extração e mantém os dados antigos completamente intactos e protegidos contra falhas de segmentação.

```

TEST(TestePaisModelo, carregarDeLinhaInvalida) {

    // Arrange: Configuração de um cenário onde a linha
não possui o separador correto

```

```

    int identificadorOriginal = 5;

    std::string nomeOriginal = "Franca";

    PaisModel paisInstanciado(identificadorOriginal,
    nomeOriginal);

    std::string linhaIncorreta =
    "TextoSemPontoEVirgula";

    // Act: Tentativa de processar a linha corrompida

    paisInstanciado.carregarDeLinha(linhaIncorreta);

    // Assert: Garantia de que o objeto rejeitou a
    leitura e manteve os dados antigos intactos

    EXPECT_EQ(paisInstanciado.obterID(),
    identificadorOriginal);

    EXPECT_EQ(paisInstanciado.obterNome(),
    nomeOriginal);

}

```

Teste do Construtor Padrão (construtorPadrao)

O que faz: Verifica se o estado inicial de um objeto da classe CidadeModel é definido de forma segura quando nenhum argumento é passado. O teste assegura que tanto os atributos herdados (id em 0, nome em "", custo polimórfico em 0.0) como o atributo exclusivo desta classe (paisAssociado em 0) iniciam completamente limpos, prevenindo falhas de dados lixo na memória.

```

TEST(TesteCidadeModelo, construtorPadrao) {

    // Arrange & Act: Criação do objeto utilizando o
    construtor sem parâmetros

    CidadeModel cidadeInstanciada;

    // Assert: Verificação se os atributos de fábrica
    iniciam zerados ou vazios

    EXPECT_EQ(cidadeInstanciada.obterID(), 0);

    EXPECT_EQ(cidadeInstanciada.obterNome(), "");

    EXPECT_EQ(cidadeInstanciada.obterIdPais(), 0);
}

```

```

        EXPECT_DOUBLE_EQ(cidadeInstanciada.obterPreco(),
0.0);

}

```

Teste do Construtor Parametrizado (construtorParametrizado)

O que faz: Valida se a classe consegue instanciar uma cidade real e mapear todos os seus dados logo no momento da criação. O teste injeta os dados por extenso do Porto (ID 25, nome "Porto", ID do país 351 e preço 85.50) e confirma, através dos métodos de leitura, se o modelo reteve e guardou todas as informações nas respectivas variáveis internas com total fidelidade.

```

TEST(TesteCidadeModelo, construtorParametrizado) {

    // Arrange: Definição dos parâmetros por extenso
    para alimentar o construtor

    int identificadorEsperado = 25;

    std::string nomeEsperado = "Porto";

    int identificadorPaisEsperado = 351;

    double precoEsperado = 85.50;

    // Act: Criação do objeto passando todas as
    variáveis configuradas

    CidadeModel
cidadeInstanciada(identificadorEsperado, nomeEsperado,
identificadorPaisEsperado, precoEsperado);

    // Assert: Validação se os métodos de leitura
    devolvem a informação correta

    EXPECT_EQ(cidadeInstanciada.obterID(),
identificadorEsperado);

    EXPECT_EQ(cidadeInstanciada.obterNome(),
nomeEsperado);

    EXPECT_EQ(cidadeInstanciada.obterIdPais(),
identificadorPaisEsperado);

    EXPECT_DOUBLE_EQ(cidadeInstanciada.obterPreco(),
precoEsperado);

}

```

Teste de Leitura de Linha Válida (carregarDeLinhaValida)

O que faz: Testa o funcionamento correto do mecanismo de divisão de texto (parsing) quando o sistema lê um registo íntegro a partir da base de dados. O teste submete a string "351;25;Porto;85.50" e valida se o método `carregarDeLinha()` consegue identificar a ordem dos dados separados por pontos e vírgulas, isolando com sucesso o ID do país, o ID da cidade, o nome e o preço correspondente.

```
TEST(TesteCidadeModelo, carregarDeLinhaValida) {

    // Arrange: Configuração do cenário com uma linha
    estruturada (pais;id;nome;preco)

    CidadeModel cidadeInstanciada;

    std::string linhaEntrada = "351;25;Porto;85.50";

    cidadeInstanciada.carregarDeLinha(linhaEntrada);

    // Assert: Garantia de que cada segmento do texto
    foi guardado no atributo correto

    EXPECT_EQ(cidadeInstanciada.obterIdPais(), 351);

    EXPECT_EQ(cidadeInstanciada.obterID(), 25);

    EXPECT_EQ(cidadeInstanciada.obterNome(), "Porto");

    EXPECT_DOUBLE_EQ(cidadeInstanciada.obterPreco(),
    85.50);

}
```

Teste de Exceção para ID Inválido (carregarComIDInvalidoLancaExcecao)

O que faz: Protege a integridade do sistema contra falhas de identificação de cidades na base de dados. O teste força a leitura de uma linha cujo ID da cidade é negativo (-5). O teste valida que o modelo reage imediatamente a esta incoerência disparando uma exceção do tipo `IDInvalidoException`, bloqueando a criação do objeto inválido.

```
TEST(TesteCidadeModelo,
carregarComIDInvalidoLancaExcecao) {

    CidadeModel cidadeInstanciada;

    std::string linhaComIdNegativo = "1;-
5;Coimbra;30.0";

    EXPECT_THROW({
```

```

        cidadeInstanciada.carregarDeLinha(linhaComIdNegativo);

        }, IDInvalidoException);

    }

```

Teste de Exceção para Nome Vazio (carregarComNomeVazioLancaExcecao)

O que faz: Garante que o catálogo não aceita destinos anónimos devido a falhas humanas ou de digitação nos ficheiros de configuração. Ao tentar carregar a linha "1;12;;30.0" (onde o espaço do nome está vazio entre os delimitadores), o teste confirma se o sistema deteta a ausência de texto e lança com sucesso um erro do tipo `ErroFormatacaoFicheiroException`.

```

TEST(TesteCidadeModelo,
carregarComNomeVazioLancaExcecao) {

    CidadeModel cidadeInstanciada;

    std::string linhaComNomeVazio = "1;12;;30.0";

    EXPECT_THROW({

        cidadeInstanciada.carregarDeLinha(linhaComNomeVazio);

        }, ErroFormatacaoFicheiroException);

}

```

Teste de Exceção para Preço Negativo (carregarComPrecoNegativoLancaExcecao)

O que faz: Aplica uma regra de negócio financeira vital no modelo de dados antes que qualquer cálculo seja efetuado pelo módulo Financeiro. O teste simula uma tentativa de leitura com um custo de viagem impossível/negativo (-15.0). O teste certifica que o sistema barra a operação de imediato lançando uma exceção do tipo `PrecoInvalidoException`.

```

TEST(TesteCidadeModelo,
carregarComPrecoNegativoLancaExcecao) {

    CidadeModel cidadeInstanciada;

    std::string linhaComPrecoNegativo = "1;15;Faro;-15.0";

    EXPECT_THROW({

        cidadeInstanciada.carregarDeLinha(linhaComPrecoNegativo);

}

```

```

    }, PrecoInvalidoException);

}

```

Teste do Construtor Padrão (construtorPadrao)

O que faz: Assegura que, ao criar uma nova instância de AtividadeModel sem passar dados iniciais, o objeto se inicializa num estado totalmente limpo e previsível. O teste valida se o identificador da atividade, o ID da cidade associada e o preço polimórfico começam todos a 0 (ou 0.0), e se a string do nome fica completamente vazia (""), protegendo o programa contra lixo de memória no arranque.

```

TEST(TesteAtividadeModelo, construtorPadrao) {

    // Arrange & Act: Criação do objeto utilizando o
    construtor sem parâmetros

    AtividadeModel atividadeInstanciada;

    // Assert: Verificação se os atributos de fábrica
    iniciam zerados ou vazios

    EXPECT_EQ(atividadeInstanciada.obterID(), 0);

    EXPECT_EQ(atividadeInstanciada.obterNome(), "");

    EXPECT_EQ(atividadeInstanciada.obterIdCidade(), 0);

    EXPECT_DOUBLE_EQ(atividadeInstanciada.obterPreco(),
    0.0);

}

```

Teste do Construtor Parametrizado (construtorParametrizado)

O que faz: Garante que a classe consegue registar uma experiência real e mapear todos os seus dados logo no momento da criação. O teste injeta os dados por extenso de um exemplo prático (ID da atividade 105, nome "Passeio de Barco", ID da cidade 25 e preço 45.00) e usa os métodos de leitura (getters) para verificar se as variáveis internas guardaram e retiveram esses atributos com total fidelidade.

```

TEST(TesteAtividadeModelo, construtorParametrizado) {

    // Arrange: Definição dos parâmetros por extenso
    para alimentar o construtor

    int identificadorEsperado = 105;

    std::string nomeEsperado = "Passeio de Barco";

    int identificadorCidadeEsperado = 25;

```

```

        double precoEsperado = 45.00;

        // Act: Criação do objeto passando todas as
        variáveis configuradas

        AtividadeModel
        atividadeInstanciada(identificadorEsperado,
        nomeEsperado, identificadorCidadeEsperado,
        precoEsperado);

        // Assert: Validação se os métodos de leitura
        devolvem a informação correta

        EXPECT_EQ(atividadeInstanciada.obterID(),
        identificadorEsperado);

        EXPECT_EQ(atividadeInstanciada.obterNome(),
        nomeEsperado);

        EXPECT_EQ(atividadeInstanciada.obterIdCidade(),
        identificadorCidadeEsperado);

        EXPECT_DOUBLE_EQ(atividadeInstanciada.obterPreco(),
        precoEsperado);

```

Teste de Leitura de Linha Válida (carregarDeLinhaValida)

O que faz: Valida o algoritmo de processamento de texto (parsing) que lida com a estrutura específica do ficheiro de atividades. Como a linha do ficheiro traz um dado extra no início que a atividade não armazena (o ID do país), o teste simula a leitura da string "351;25;105;Passeio de Barco;45.00". O objetivo é certificar que o método carregarDeLinha() consegue saltar com sucesso o ID do país (351) e extrair de forma correta o ID da cidade, o ID da atividade, o nome e o preço.

```

TEST(TesteAtividadeModelo, carregarDeLinhaValida) {

    // Arrange: Configuração de uma linha estruturada
    (pais;cidade;atividade;nome;preco)

    // Nota: O código da atividade deve ignorar o
    primeiro valor (351)

    AtividadeModel atividadeInstanciada;

    std::string linhaEntrada = "351;25;105;Passeio de
    Barco;45.00";

    atividadeInstanciada.carregarDeLinha(linhaEntrada);

```



```

        // Assert: Garantia de que o salto do país
        funcionou e os dados foram guardados

        EXPECT_EQ(atividadeInstanciada.obterIdCidade(),
25);

        EXPECT_EQ(atividadeInstanciada.obterID(), 105);

        EXPECT_EQ(atividadeInstanciada.obterNome(),
"Passeio de Barco");

        EXPECT_DOUBLE_EQ(atividadeInstanciada.obterPreco(),
45.00);

    }

```

Teste de Exceção para Preço Negativo (carregarComPrecoNegativoLancaExcecao)
 O que faz: Funciona como uma barreira de segurança financeira no modelo de dados, impedindo que bilhetes ou tickets com valores monetários absurdos entrem no sistema. Ao tentar forçar o carregamento de uma linha de texto com um custo negativo (-5.0), o teste confirma se o sistema deteta a irregularidade imediatamente e dispara a exceção `PrecoInvalidoException`, bloqueando a criação da atividade corrompida.

```

TEST(TesteAtividadeModelo,
carregarComPrecoNegativoLancaExcecao) {

    AtividadeModel atividadeInstanciada;

    std::string linhaInvalida = "1;5;101;Passeio de
Metro;-5.0";

    EXPECT_THROW({

        atividadeInstanciada.carregarDeLinha(linhaInvalida);

    }, PrecoInvalidoException);

}

```

Teste do Construtor Padrão (construtorPadrao)

O que faz: Garante que, ao criar o gestor central do catálogo (`RepositorioCatalogo`), a estrutura interna de armazenamento é alocada de forma limpa e segura. O teste valida que as coleções (vetores) que vão guardar os países, as cidades e as atividades começam completamente vazias (`empty()`), assegurando que o sistema está pronto para receber novos dados sem herdar lixo de memória.

```

TEST(TesteRepositorioCatalogo, construtorPadrao) {

    // Arrange & Act: Criação da instância do
    repositório

    RepositorioCatalogo repositorioInstanciado;

    // Assert: Confirmação de que as listas começam
    completamente vazias e seguras

    EXPECT_TRUE(repositorioInstanciado.obterPaises().empty());

    EXPECT_TRUE(repositorioInstanciado.obterCidades().empty());

    EXPECT_TRUE(repositorioInstanciado.obterAtividades().empty());

}

```

Teste de Exceção para Ficheiros Inexistentes (carregarFicheirosInexistentesLancaExcecao)

O que faz: Avalia a robustez e a tolerância a falhas do repositório caso ocorra um problema crítico no sistema de ficheiros (por exemplo, se os ficheiros de texto que servem de base de dados forem apagados, renomeados ou movidos por acidente). O teste força a chamada do método carregarTudo() sem que os ficheiros estejam presentes, confirmando se o programa reage de forma controlada disparando a exceção ErroFormatacaoFicheiroException em vez de simplesmente falhar e fechar sozinho (crashar).

```

TEST(TesteRepositorioCatalogo,
carregarFicheirosInexistentesLancaExcecao) {

    // Arrange: Preparação do repositório

    RepositorioCatalogo repositorioInstanciado;

    // Act & Assert: Garantir que o sistema lança a
    exceção correta se os ficheiros sumirem

    EXPECT_THROW({

        repositorioInstanciado.carregarTudo();

    }, ErroFormatacaoFicheiroException);

}

```

Teste do Construtor Padrão (construtorPadrao)

O que faz: Verifica se o carrinho de compras do utilizador (o roteiro de viagens) é criado de forma limpa e sem dados prévios. O teste valida se a lista de escolhas (obterEscolhas()) nasce completamente vazia, assegurando que o cliente começa a planear o seu itinerário do zero.

```
TEST(TesteRoteiroModelo, construtorPadrao) {

    // Arrange & Act: Criação da instância principal do
    roteiro

    RoteiroModel roteiroInstanciado;

    // Assert: Confirmação de que a lista de escolhas
    nasce completamente vazia

    EXPECT_TRUE(roteiroInstanciado.obterEscolhas().empty());

}
```

Teste de Adição de Escolha Válida (adicionarEscolhaValida)

O que faz: Simula o comportamento do cliente ao adicionar um destino ou atividade ao seu plano de viagem. O teste utiliza uma classe auxiliar de simulação (TesteLocalidade) para inserir "Coimbra" no roteiro e valida se a estrutura dinâmica armazena o ponteiro do objeto com sucesso, aumentando o tamanho da lista para 1 e retendo o nome correto do destino.

```
TEST(TesteRoteiroModelo, adicionarEscolhaValida) {

    // Arrange: Preparação do roteiro e de uma
    localidade simulada

    RoteiroModel roteiroInstanciado;

    TesteLocalidade localidadeTeste(10, "Coimbra",
    25.50);

    // Act: Inserção da localidade dentro da lista de
    escolhas

    roteiroInstanciado.adicionarEscolha(&localidadeTeste);

    // Assert: Verificação de que a lista aumentou e
    contém o item correto
```

```

        EXPECT_EQ(roteiroInstanciado.obterEscolhas().size()
, 1);

        EXPECT_EQ(roteiroInstanciado.obterEscolhas()[0]-
>obterNome(), "Coimbra");

    }

```

Teste de Adição de Escolha Nula (adicionarEscolhaNula)

O que faz: Funciona como uma proteção contra erros de memória e ponteiros perdidos (null pointers). Caso o sistema tente adicionar um elemento que não existe ou falhe ao procurar um ID, este teste garante que o método adicionarEscolha rejeita de forma segura o ponteiro nullptr (nulo), impedindo que o roteiro fique corrompido e mantendo a lista vazia.

```

TEST(TesteRoteiroModelo, adicionarEscolhaNula) {

    // Arrange: Preparação do roteiro e de um ponteiro
    vazio (para simular um erro)

    RoteiroModel roteiroInstanciado;

    LocalidadeModel* ponteiroNulo = nullptr;

    // Act: Tentativa de adicionar o ponteiro vazio à
    lista

    roteiroInstanciado.adicionarEscolha(ponteiroNulo);

    // Assert: Garantia de que a função deteta o nulo e
    não adiciona nada à lista

    EXPECT_TRUE(roteiroInstanciado.obterEscolhas().empty());

}

```

Teste de Esvaziamento de Roteiro (esvaziarRoteiro)

O que faz: Valida a funcionalidade de "limpar carrinho" ou "cancelar plano atual". O teste preenche o roteiro com uma localização intermédia ("Braga") e, logo de seguida, aciona o método esvaziar(). O objetivo é certificar que todos os elementos acumulados são descartados e que a lista regressa com sucesso ao estado inicial de zero elementos.

```

TEST(TesteRoteiroModelo, esvaziarRoteiro) {

    // Arrange: Criação de um roteiro preenchido com um
    item

    RoteiroModel roteiroInstanciado;

```

```

        TesteLocalidade localidadeTeste(5, "Braga", 15.00);

        roteiroInstanciado.adicionarEscolha(&localidadeTeste);

        // Act: Acionamento do mecanismo de limpeza da lista

        roteiroInstanciado.esvaziar();

        // Assert: Confirmação de que o roteiro voltou ao estado vazio

        EXPECT_TRUE(roteiroInstanciado.obterEscolhas().empty());
    }

```

Teste de Gravação de Ficheiro Vazio (gravarFicheiroVazio)

O que faz: Aplica uma regra de negócio que impede a criação de reservas fantasmas ou sem conteúdo associado. O teste tenta exportar e gravar um ficheiro de reserva passando todos os metadados necessários (ID, estado, descrição, datas, etc.), mas com o roteiro totalmente vazio. O teste valida que o sistema aborta a operação e devolve false.

```

TEST(TesteRoteiroModelo, gravarFicheiroVazio) {

    RoteiroModel roteiroInstanciado;

    // Act: Passar os 9 parâmetros exigidos pela nova assinatura do Modelo

    bool resultadoGravacao =
roteiroInstanciado.gravarFicheiroReserva(

        1, "Pendente", "Viagem de Teste", "Sem notas",

        "12/06/2026", "15/06/2026", 3, 0.0, 0.0

    );

    // Assert: Como o roteiro está vazio, deve retornar falso

    EXPECT_FALSE(resultadoGravacao);
}

```

Teste de Gravação de Ficheiro Preenchido (gravarFicheiroPreenchido)

O que faz: Garante a persistência de dados e a criação bem-sucedida do ficheiro de reserva final (a "fatura/resumo" da viagem). O teste adiciona um destino ao roteiro ("Faro") e executa o método `gravarFicheiroReserva` enviando os 9 parâmetros obrigatórios (incluindo as datas de ida e volta e os custos totais processados). O teste confirma se o método conclui a operação devolvendo `true`.

```
TEST(TesteRoteiroModelo, gravarFicheiroPreenchido) {  
  
    RoteiroModel roteiroInstanciado;  
  
    TesteLocalidade localidadeTeste(1, "Faro", 100.0);  
  
    roteiroInstanciado.adicionarEscolha(&localidadeTeste);  
  
    // Act: Passar os 9 parâmetros exigidos  
  
    bool resultadoGravacao =  
        roteiroInstanciado.gravarFicheiroReserva(  
  
            1, "Confirmada", "Ferias de Verao", "Levar  
protetor",  
  
            "12/06/2026", "19/06/2026", 7, 100.0, 123.0  
  
        );  
  
    // Assert  
  
    EXPECT_TRUE(resultadoGravacao);  
  
}
```

Teste de Lista Vazia (`calcularTotalBaseListaVazia`)

O que faz: Garante que a calculadora financeira inicia a contagem a zeros e não gera valores fantasmas. O teste passa um vetor de localidades completamente vazio para o método estático `calcularTotalBase` e valida se o retorno é exatamente 0.0, certificando que o sistema lida corretamente com a ausência de compras no roteiro.

```
TEST(TesteFinanceiroModelo,  
calcularTotalBaseListaVazia) {  
  
    // Arrange: Preparação de uma lista completamente  
vazia  
  
    std::vector<LocalidadeModel*>  
listaLocalidadesVazia;
```

```

        // Act: Execução do cálculo estático

        double resultadoCalculado =
Financeiro::calcularTotalBase(listaLocalidadesVazia);

        // Assert: O total de uma lista vazia deve ser zero
        vírgula zero

        EXPECT_DOUBLE_EQ(resultadoCalculado, 0.0);

    }

```

Teste com Múltiplos Elementos (calcularTotalBaseComElementos)

O que faz: Valida o algoritmo principal de acumulação financeira (somatório). O teste injeta duas localidades simuladas na lista — uma com o preço de 50.0 e outra com 150.0 — e verifica se o motor financeiro consegue percorrer o vetor, extrair os preços individuais através de polimorfismo e somá-los com precisão, totalizando 200.0.

```

TEST(TesteFinanceiroModelo,
calcularTotalBaseComElementos) {

    // Arrange: Criação de localidades com preços
    distintos e inserção na lista

    TesteLocalidade primeiraLocalidade(1, "Porto",
50.0);

    TesteLocalidade segundaLocalidade(2, "Lisboa",
150.0);

    std::vector<LocalidadeModel*>
listaLocalidadesPreenchida;

    listaLocalidadesPreenchida.push_back(&primeiraLocal
idade);

    listaLocalidadesPreenchida.push_back(&segundaLocal
idade);

    // Act: Execução do cálculo da soma total acumulada

    double resultadoCalculado =
Financeiro::calcularTotalBase(listaLocalidadesPreenchid
a);

    // Assert: Verificação se a soma corresponde
    exatamente ao esperado (50.0 + 150.0 = 200.0)

```

```
EXPECT_DOUBLE_EQ(resultadoCalculado, 200.0);
```

```
}
```

Teste de Segurança com Ponteiro Nulo (calcularTotalBaseComPonteiroNulo)

O que faz: Avalia a resiliência do motor de cálculo contra falhas de ponteiros perdidos na lista de escolhas. Ao introduzir propositadamente um ponteiro válido (30.0) e um ponteiro nulo (nullptr) no mesmo vetor, este teste certifica que a função ignora o elemento nulo com segurança em vez de tentar aceder-lhe na memória (o que causaria o fecho imediato da aplicação), devolvendo com sucesso apenas o valor do item íntegro (30.0).

```
TEST(TesteFinanceiroModelo,
calcularTotalBaseComPonteiroNulo) {

    // Arrange: Configuração de uma lista que contém um
    // ponteiro nulo para testar a segurança

    TesteLocalidade primeiraLocalidade(1, "Braga",
30.0);

    LocalidadeModel* ponteiroNulo = nullptr;

    std::vector<LocalidadeModel*>
listaLocalidadesComNulo;

    listaLocalidadesComNulo.push_back(&primeiraLocalida
de);

    listaLocalidadesComNulo.push_back(ponteiroNulo);

    double resultadoCalculado =
Financeiro::calcularTotalBase(listaLocalidadesComNulo);

    // Assert: Garantia de que o código ignora o nulo
    // com segurança e soma apenas o valor válido

    EXPECT_DOUBLE_EQ(resultadoCalculado, 30.0);

}
```

Teste de Cálculo do Valor do IVA (calcularValorIva)

O que faz: Testa isoladamente a regra de imposto aplicada pelo sistema. Tendo como base um valor redondo de 100.0, o método calcularValorIVA é executado para extrair a fatia correspondente à taxa legal. O teste valida se o retorno é exatamente 23.0, confirmando que a lógica interna aplica rigorosamente a taxa de 23% de IVA.

```
TEST(TesteFinanceiroModelo, calcularValorIva) {
```



```

        // Arrange: Definição de um valor base redondo para
        facilitar a conta matemática

        double valorBaseTeste = 100.0;

        double valorIvaEsperado = 23.0;

        // Act: Execução do cálculo isolado da taxa de
        imposto

        double resultadoCalculado =
        Financeiro::calcularValorIVA(valorBaseTeste);

        // Assert: Confirmação de que a taxa aplicada foi
        exatamente de vinte e três por cento

        EXPECT_DOUBLE_EQ(resultadoCalculado,
        valorIvaEsperado);

    }

```

Teste do Custo Total com IVA (calcularTotalComIva)

O que faz: Certifica que a fórmula matemática do preço final que será cobrado ao cliente está correta. O teste submete um valor base de 100.0 e valida se o método calcularTotalComIVA consegue somar o custo original com o imposto calculado, devolvendo o montante final correto de 123.0.

```

TEST(TesteFinanceiroModelo, calcularTotalComIva) {

    // Arrange: Configuração do cenário de teste para o
    montante final amealhado

    double valorBaseTeste = 100.0;

    double valorTotalEsperado = 123.0;

    // Act: Execução do cálculo do preço final
    acrescido do imposto

    double resultadoCalculado =
    Financeiro::calcularTotalComIVA(valorBaseTeste);

    // Assert: Validação matemática do resultado obtido

    EXPECT_DOUBLE_EQ(resultadoCalculado,
    valorTotalEsperado);

}

```

Teste de Configuração do Método de Pagamento (gerirMetodosDePagamento)

O que faz: Valida a máquina de estados estática que dita como a reserva será liquidada. O teste altera o método de pagamento para CARTAO e verifica se o sistema retém esse estado e se consegue traduzi-lo corretamente para formato de texto ("Cartao"). Logo a seguir, altera para DINHEIRO e repete a validação, garantindo que a alternância entre opções no menu funciona na perfeição e sem misturar os dados.

```
TEST(TesteFinanceiroModelo, gerirMetodosDePagamento) {

    // Act 1: Definir como Cartão

    Financeiro::definirMetodo(MetodoPagamento::CARTAO);

    // Assert 1: Validar armazenamento e conversão
    textual

    EXPECT_EQ(Financeiro::obterMetodoSelecioneado(),
MetodoPagamento::CARTAO);

    EXPECT_EQ(Financeiro::obterMetodoPagamentoComoStrin
g(), "Cartão");

    // Act 2: Alterar para Dinheiro

    Financeiro::definirMetodo(MetodoPagamento::DINHEIRO
);

    // Assert 2: Validar a alteração do estado estático

    EXPECT_EQ(Financeiro::obterMetodoSelecioneado(),
MetodoPagamento::DINHEIRO);

    EXPECT_EQ(Financeiro::obterMetodoPagamentoComoStrin
g(), "Dinheiro");

}
```

Teste ao Construtor: Datas Válidas (Caminho Feliz)

Explicação: Este teste verifica o cenário normal e ideal (Caminho Feliz) onde o utilizador insere uma data de chegada e uma data de saída perfeitamente normais e válidas (do dia 1 ao dia 10 de junho de 2024). O objetivo é garantir que o objeto Data é criado com sucesso e que nenhuma exceção é atirada. Como nenhuma exceção ocorre, a variável flag mantém-se false, validando o teste (EXPECT_FALSE).

```
TEST(DataConstructorTest, DatasValidas) {

    bool flag = false;
```

```

        try {

            Data d(1, 6, 2024, 10, 6, 2024);

        } catch (DataInvalidaException& e) {

            flag = true;

        }

        EXPECT_FALSE(flag);

    }

```

Teste ao Construtor: Mês Inválido

Explicação: Este teste avalia a robustez do programa contra erros de introdução do utilizador. Como o nosso calendário só tem 12 meses, tentar criar uma viagem com os meses "13" e "14" viola as regras da validação. O sistema deteta o erro e lança a exceção `DataInvalidaException`, ativando a flag para true (`EXPECT_TRUE`).

```

TEST(DataConstructorTest, MesInvalido) {

    bool flag = false;

    try {

        Data d(1, 13, 2024, 10, 14, 2024);

    } catch (DataInvalidaException& e) {

        flag = true;

    }

    EXPECT_TRUE(flag);

}

```

Teste ao Construtor: Dia Inválido

Explicação: Semelhante ao teste do mês, aqui testamos os limites dos dias do mês. São inseridos os dias 32 e 33, que não existem em nenhum mês do ano. O construtor deve rejeitar estes valores e atirar a devida exceção.

```

TEST(DataConstructorTest, DiaInvalido) {

    bool flag = false;

    try {

```

```

        Data d(32, 6, 2024, 33, 6, 2024);

    } catch (DataInvalidaException& e) {

        flag = true;

    }

    EXPECT_TRUE(flag);

}

```

Teste ao Construtor: Ano Anterior a 2024

Explicação: Este teste valida uma regra de negócio muito específica da aplicação: não é permitido agendar viagens para o passado. Como o ano base estabelecido é 2024, tentar criar um registro para o ano 2023 tem de falhar obrigatoriamente.

```

TEST(DataConstructorTest, AnoAnteriorA2024) {

    bool flag = false;

    try {

        Data d(1, 6, 2023, 10, 6, 2023);

    } catch (DataInvalidaException& e) {

        flag = true;

    }

    EXPECT_TRUE(flag);

}

```

Teste ao Construtor: Inconsistência Cronológica (Saída Antes da Chegada)

Explicação: Aqui testamos a lógica cronológica do objeto temporal. Tenta-se criar uma viagem onde a data de chegada é dia 10, mas a data de saída é dia 1 do mesmo mês. É impossível terminar uma viagem antes de a começar, logo o sistema tem de rejeitar e atirar a exceção.

```

TEST(DataConstructorTest, SaidaAntesDeChegada) {

    bool flag = false;

    try {

        Data d(10, 6, 2024, 1, 6, 2024);

    } catch (DataInvalidaException& e) {

        flag = true;

    }

    EXPECT_TRUE(flag);

}

```

```

    } catch (DataInvalidaException& e) {

        flag = true;

    }

    EXPECT_TRUE(flag);

}

```

Teste ao Construtor: Viagem de Um Só Dia

Explicação: Avalia um caso limite (Edge Case) aceitável. Se o utilizador agendar uma viagem cuja chegada e saída ocorrem exatamente no mesmo dia (ex: 5/6/2024), isso é um cenário válido. O teste garante que o programa não bloqueia e permite a criação do objeto (EXPECT_FALSE).

```

TEST(DataConstructorTest, MesmoDia) {

    bool flag = false;

    try {

        Data d(5, 6, 2024, 5, 6, 2024);

    } catch (DataInvalidaException& e) {

        flag = true;

    }

    EXPECT_FALSE(flag);

}

```

Teste ao Construtor: Ano Bissexto Válido

Explicação: Este teste demonstra a capacidade avançada de validação de calendário da classe Data. O ano de 2024 é um ano bissexto, o que significa que fevereiro tem 29 dias. Ao inserir "29 de Fevereiro de 2024", o programa deve reconhecer a data como válida e não atirar erro.

```

TEST(DataConstructorTest, FevereiroBissextoValido) {

    bool flag = false;

    try {

        Data d(29, 2, 2024, 1, 3, 2024);

    } catch (DataInvalidaException& e) {

    }
}

```

```

        flag = true;
    }

    EXPECT_FALSE(flag);
}

```

Teste ao Construtor: Ano Não-Bissexto Inválido

Explicação: Em contraste direto com o teste anterior, este tenta agendar uma viagem para o dia 29 de Fevereiro num ano comum (2025). Como fevereiro de 2025 só tem 28 dias, a classe tem de detetar a inconsistência e atirar a exceção de erro.

```

TEST(DataConstructorTest, FevereiroNaoBissextoInvalido)
{
    bool flag = false;

    try {
        Data d(29, 2, 2025, 1, 3, 2025);
    } catch (DataInvalidaException& e) {
        flag = true;
    }

    EXPECT_TRUE(flag);
}

```

Teste ao Construtor: Limites Finais dos Meses (Dia 31 num Mês de 30)

Explicação: Mais um teste focado no conhecimento interno do calendário. O mês 4 (Abril) tem apenas 30 dias. O teste tenta criar uma data para o dia "31 de Abril", garantindo que a regra restritiva atira a devida exceção de invalidação.

```

TEST(DataConstructorTest, Dia31EmMesComSo30Dias) {
    bool flag = false;

    try {
        Data d(31, 4, 2024, 31, 5, 2024);
    } catch (DataInvalidaException& e) {
        flag = true;
    }
}

```

```

    }

    EXPECT_TRUE(flag);

```

```

}

```

Teste da Lógica de Negócio: Cálculo de Dias para Mesmo Dia

Explicação: Este teste foca-se no método `getNumeroDias()`. Se a data de chegada e a data de saída forem exatamente o mesmo dia, o cálculo matemático interno da duração da viagem tem de ser rigorosamente zero (`EXPECT_EQ(dias, 0)`).

```

TEST(DataNumeroDiasTest, MesmoDiaZeroDias) {

    Data d(5, 6, 2024, 5, 6, 2024);

    int dias = d.getNumeroDias();

    EXPECT_EQ(dias, 0);

}

```

Teste da Lógica de Negócio: Cálculo de Dias (Uma Semana)

Explicação: Valida a capacidade de calcular a duração de uma viagem linear dentro do mesmo mês. Iniciar a dia 1 e sair a dia 8 resulta numa diferença matemática esperada de 7 dias inteiros.

```

TEST(DataNumeroDiasTest, UmaSemana) {

    Data d(1, 6, 2024, 8, 6, 2024);

    int dias = d.getNumeroDias();

    EXPECT_EQ(dias, 7);

}

```

Teste da Lógica de Negócio: Cálculo de Dias (Transição de Mês)

Explicação: Este é o teste matemático mais crítico da classe. Verifica se o algoritmo de contagem de dias funciona através das "fronteiras" dos meses. Indo de 28 de Junho (mês de 30 dias) a 2 de Julho, o programa tem de saber compensar a mudança de mês e devolver o valor correto de 4 dias.

```

TEST(DataNumeroDiasTest, PassagemDeMes) {

    Data d(28, 6, 2024, 2, 7, 2024);

    int dias = d.getNumeroDias();

    EXPECT_EQ(dias, 4);

}

```

```
}
```

Teste ao Setter: Alterar Chegada com Sucesso

Explicação: Verifica se, após criar uma viagem, é possível modificar livremente a data de chegada através do `setChegada`, desde que a nova data inserida também seja válida (passar de dia 1 para dia 2 do mesmo mês).

```
TEST(DataSetChegadaTest, DataValida) {  
  
    Data d(1, 6, 2024, 10, 6, 2024);  
  
    bool flag = false;  
  
    try {  
  
        d.setChegada(2, 6, 2024);  
  
    } catch (DataInvalidaException& e) {  
  
        flag = true;  
  
    }  
  
    EXPECT_FALSE(flag);  
  
}
```

Teste ao Setter: Proteção contra Data de Chegada Inválida

Explicação: Garante que o *setter* não é vulnerável a dados corrompidos. Tentar alterar a data de chegada em memória para o dia inexistente "32" ativa os mesmos mecanismos de defesa que o construtor, atirando uma exceção para proteger o estado do objeto.

```
TEST(DataSetChegadaTest, DataInvalida) {  
  
    Data d(1, 6, 2024, 10, 6, 2024);  
  
    bool flag = false;  
  
    try {  
  
        d.setChegada(32, 6, 2024);  
  
    } catch (DataInvalidaException& e) {  
  
        flag = true;  
  
    }  
  
    EXPECT_TRUE(flag);  
  
}
```



```
}
```

Teste ao Setter: Chegada Depois da Saída (Corrupção de Estado)

Explicação: Este teste assegura que não podemos manipular um *setter* para criar uma impossibilidade cronológica. Se a saída já está agendada para o dia 10, alterar o início (chegada) para o dia 15 faria com que a viagem tivesse dias negativos. A exceção bloqueia esta alteração perigosa.

```
TEST(DataSetChegadaTest, ChegadaDepoisDaSaida) {  
  
    Data d(1, 6, 2024, 10, 6, 2024);  
  
    bool flag = false;  
  
    try {  
  
        d.setChegada(15, 6, 2024);  
  
    } catch (DataInvalidaException& e) {  
  
        flag = true;  
  
    }  
  
    EXPECT_TRUE(flag);  
  
}
```

Teste ao Setter: Alterar Saída com Sucesso

Explicação: Valida o método `setSaida`. Modifica de forma segura a data de término da viagem prolongando-a para um dia posterior (dia 15). O estado atualiza-se corretamente sem que ocorram exceções.

```
TEST(DataSetSaidaTest, DataValida) {  
  
    Data d(1, 6, 2024, 10, 6, 2024);  
  
    bool flag = false;  
  
    try {  
  
        d.setSaida(15, 6, 2024);  
  
    } catch (DataInvalidaException& e) {  
  
        flag = true;  
  
    }  
  
}
```

```
EXPECT_FALSE(flag);
```

```
}
```

Teste ao Setter: Proteção contra Data de Saída Inválida

Explicação: Semelhante ao teste 14, protege a classe contra valores lixo passados pelo utilizador ou por erro de sistema. Tentar agendar a saída para um dia "0" falha redondamente.

```
TEST(DataSetSaidaTest, DataInvalida) {  
  
    Data d(1, 6, 2024, 10, 6, 2024);  
  
    bool flag = false;  
  
    try {  
  
        d.setSaida(0, 6, 2024);  
  
    } catch (DataInvalidaException& e) {  
  
        flag = true;  
  
    }  
  
    EXPECT_TRUE(flag);  
  
}
```

Teste ao Setter: Antecipar Saída Invalida a Lógica Temporal

Explicação: Tenta adulterar o fim da viagem (saída) para uma data anterior à data de início já estabelecida (dia 1 versus dia 5). O setSaida recalcula a duração, nota o absurdo cronológico e lança a exceção de invalidação (EXPECT_TRUE).

```
TEST(DataSetSaidaTest, SaidaAntesDeChegada) {  
  
    Data d(5, 6, 2024, 10, 6, 2024);  
  
    bool flag = false;  
  
    try {  
  
        d.setSaida(1, 6, 2024);  
  
    } catch (DataInvalidaException& e) {  
  
        flag = true;  
  
    }  
  
}
```

```
EXPECT_TRUE(flag);
```

```
}
```

Teste à Estrutura Dinâmica: Adicionar um Horário

Explicação: Foca-se na lista (vetor dinâmico) de atividades guardada dentro do objeto Data. Ao usar addHorario(), insere-se uma *string*, sendo esperado que a contagem (size()) da lista passe a registrar exatamente 1 elemento.

```
TEST(DataHorariosTest, AdicionarUmHorario) {
```

```
    Data d(1, 6, 2024, 10, 6, 2024);
```

```
    d.addHorario("09:00 - Visita ao museu");
```

```
    EXPECT_EQ(d.getHorarios().size(), 1);
```

```
}
```

Teste à Estrutura Dinâmica: Adicionar Múltiplos Horários

Explicação: Verifica a capacidade de a classe escalar e reter vários dados em sequência sem se corromper. Ao adicionar 3 atividades sequencialmente, o tamanho da lista interna tem de refletir com exatidão os 3 registos guardados (EXPECT_EQ(..., 3)).

```
TEST(DataHorariosTest, AdicionarVariosHorarios) {
```

```
    Data d(1, 6, 2024, 10, 6, 2024);
```

```
    d.addHorario("09:00 - Pequeno almoco");
```

```
    d.addHorario("11:00 - Visita ao museu");
```

```
    d.addHorario("14:00 - Almoco");
```

```
    EXPECT_EQ(d.getHorarios().size(), 3);
```

```
}
```

Teste de Estado Inicial: Sem Horários Definidos

Explicação: Valida o construtor do ponto de vista das estruturas de dados (vetores). Logo após uma viagem nova ser instanciada, a sua lista de horários deve estar completamente limpa e sem elementos residuais. O teste exige que o método .empty() seja estritamente verdadeiro.

```
TEST(DataHorariosTest, SemHorarios) {
```

```
    Data d(1, 6, 2024, 10, 6, 2024);
```

```
    EXPECT_TRUE(d.getHorarios().empty());
```

```
}
```

Teste de Valores Iniciais do Título e Notas

Este teste verifica o estado inicial do modelo. Garante que, ao criar um novo objeto sem parâmetros, o título assume o valor padrão de fábrica e as notas iniciam completamente vazias.

```
TEST(TituloNotasTest, ValoresIniciais) {  
  
    // Arrange  
  
    Titulo_notas_modelo modelo;  
  
    // Act & Assert  
  
    EXPECT_EQ(modelo.getTitulo(), "Sem título");  
  
    EXPECT_EQ(modelo.getNotasViagem(), "");  
  
}
```

Teste de Atribuição de Título Válido

Valida o comportamento do sistema perante a inserção de um título de viagem preenchido corretamente. O teste confirma que a alteração é registrada na memória e que a função devolve o valor booleano verdadeiro.

```
TEST(TituloNotasTest,  
SetTituloValidoFuncionaERetornaTrue) {  
  
    // Arrange  
  
    Titulo_notas_modelo modelo;  
  
    // Act  
  
    bool resultado = modelo.setTitulo("Viagem a  
Paris");  
  
    // Assert  
  
    EXPECT_TRUE(resultado);  
  
    EXPECT_EQ(modelo.getTitulo(), "Viagem a Paris");  
  
}
```

Teste de Título Abaixo do Limite Mínimo

Garante que títulos com comprimento igual ou inferior a dois caracteres sejam rejeitados pelo sistema de validação, mantendo o valor padrão guardado sem sofrer modificações.

```
TEST(TituloNotasTest, SetTituloMuitoCurtoFalha) {

    // Arrange

    Titulo_notas_modelo modelo;

    // Act

    bool resultado = modelo.setTitulo("ab"); // length
    == 2, <= 2 -> inválido

    // Assert

    EXPECT_FALSE(resultado);

    EXPECT_EQ(modelo.getTitulo(), "Sem título"); // não
    foi alterado

}
```

Teste de Atribuição de Título Vazio

Certifica que uma string totalmente vazia é tratada como uma entrada inválida pelas regras de negócio da aplicação, bloqueando a operação de alteração.

```
TEST(TituloNotasTest, SetTituloVazioFalha) {

    // Arrange

    Titulo_notas_modelo modelo;

    // Act

    bool resultado = modelo.setTitulo("");

    // Assert

    EXPECT_FALSE(resultado);

}
```

Teste de Título Acima do Limite Máximo

Avalia a imposição do limite máximo de caracteres para o título da viagem, forçando a rejeição de strings com comprimento igual ou superior a vinte caracteres.

```
TEST(TituloNotasTest, SetTituloMuitoLongoFalha) {
```

```

// Arrange

Titulo_notas_modelo modelo;

std::string tituloLongo(20, 'a'); // length == 20,
>= 20 -> inválido

// Act

bool resultado = modelo.setTitulo(tituloLongo);

// Assert

EXPECT_FALSE(resultado);

EXPECT_EQ(modelo.getTitulo(), "Sem título");

}

```

Teste de Fronteira do Limite Mínimo do Título

Testa o limite inferior exato de aceitação da regra de validação, provando que uma string com três caracteres (limite de fronteira mínimo válido) é integrada no sistema com sucesso.

```

TEST(TituloNotasTest, SetTituloLimiteMinimoValido) {

// Arrange

Titulo_notas_modelo modelo;

std::string titulo(3, 'a'); // length == 3, > 2 ->
válido

// Act

bool resultado = modelo.setTitulo(titulo);

// Assert

EXPECT_TRUE(resultado);

EXPECT_EQ(modelo.getTitulo(), titulo);

}

```

Teste de Fronteira do Limite Máximo do Título

Testa o limite superior exato da regra de validação, comprovando que uma string com dezanove caracteres (limite máximo de caracteres permitido) é guardada na estrutura de dados do objeto.

```
TEST(TituloNotasTest, SetTituloLimiteMaximoValido) {

    // Arrange

    Titulo_notas_modelo modelo;

    std::string titulo(19, 'a'); // length == 19, < 20
-> válido

    // Act

    bool resultado = modelo.setTitulo(titulo);

    // Assert

    EXPECT_TRUE(resultado);

    EXPECT_EQ(modelo.getTitulo(), titulo);

}
```

Teste de Atribuição de Notas Válidas

Confirma a inserção correta de notas auxiliares válidas na viagem, verificando se a string correspondente é guardada na íntegra e se o método retorna a confirmação de sucesso.

```
TEST(TituloNotasTest,
SetNotasValidasFuncionaERetornaTrue) {

    // Arrange

    Titulo_notas_modelo modelo;

    // Act

    bool resultado = modelo.setNotasViagem("Levar
protetor solar e passaporte.");

    // Assert

    EXPECT_TRUE(resultado);

    EXPECT_EQ(modelo.getNotasViagem(), "Levar protetor
solar e passaporte.");

}
```

```
}
```

Teste de Notas Abaixo do Limite Mínimo

Verifica se o sistema rejeita notas de viagem excessivamente curtas, com comprimento inferior ou igual a dois caracteres, preservando o estado e o conteúdo original intacto.

```
TEST(TituloNotasTest, SetNotasMuitoCurtasFalha) {  
  
    // Arrange  
  
    Titulo_notas_modelo modelo;  
  
    // Act  
  
    bool resultado = modelo.setNotasViagem("ok"); //  
length == 2, <= 2 -> inválido  
  
    // Assert  
  
    EXPECT_FALSE(resultado);  
  
    EXPECT_EQ(modelo.getNotasViagem(), ""); // não foi  
alterado  
  
}
```

Teste de Notas Acima do Limite Máximo

Testa a restrição de armazenamento máximo do buffer de notas, forçando uma falha na validação caso o texto descritivo atinja ou ultrapasse o limite crítico de quinhentos caracteres.

```
TEST(TituloNotasTest, SetNotasMuitoLongasFalha) {  
  
    // Arrange  
  
    Titulo_notas_modelo modelo;  
  
    std::string notasLongas(500, 'a'); // length ==  
500, >= 500 -> inválido  
  
    // Act  
  
    bool resultado =  
modelo.setNotasViagem(notasLongas);  
  
    // Assert  
  
    EXPECT_FALSE(resultado);  
  
}
```



```
EXPECT_EQ(modelo.getNotasViagem(), "");
```

```
}
```

Teste de Fronteira do Limite Mínimo de Notas

Valida a fronteira inferior do campo das notas de viagem, confirmando que uma string com exatamente três caracteres de comprimento é aceite sem disparar bloqueios.

```
TEST(TituloNotasTest, SetNotasLimiteMinimoValido) {  
  
    // Arrange  
  
    Titulo_notas_modelo modelo;  
  
    std::string notas(3, 'a'); // length == 3, > 2 ->  
válido  
  
    // Act  
  
    bool resultado = modelo.setNotasViagem(notas);  
  
    // Assert  
  
    EXPECT_TRUE(resultado);  
  
    EXPECT_EQ(modelo.getNotasViagem(), notas);  
  
}
```

Teste de Fronteira do Limite Máximo de Notas

Valida a fronteira superior das notas de viagem, assegurando que um texto longo com quatrocentos e noventa e nove caracteres é processado e armazenado de forma estável.

```
TEST(TituloNotasTest, SetNotasLimiteMaximoValido) {  
  
    // Arrange  
  
    Titulo_notas_modelo modelo;  
  
    std::string notas(499, 'a'); // length == 499, <  
500 -> válido  
  
    // Act  
  
    bool resultado = modelo.setNotasViagem(notas);  
  
    // Assert  
  
    EXPECT_TRUE(resultado);  
  
}
```

```
EXPECT_EQ(modelo.getNotasViagem(), notas);
```

```
}
```

Teste de Estado Inicial do Número de Reserva

Certifica que o gerador de identificadores sequenciais inicia o contador na unidade base padrão, estabelecendo o ponto de partida numérico correto para o registo de novas reservas.

```
TEST(NumeroReservaTest, ValorInicialEhUm) {
```

```
    // Arrange
```

```
    Numero_reserva_modelo modelo;
```

```
    // Act
```

```
    int numero = modelo.getNumeroReserva();
```

```
    // Assert
```

```
    EXPECT_EQ(numero, 1);
```

```
}
```

Teste de Incremento Unitário da Reserva

Avalia o comportamento operacional básico do acumulador incremental, garantindo que o número identificador da reserva sobe exatamente uma unidade após sofrer uma chamada de atualização.

```
TEST(NumeroReservaTest, IncrementarAumentaUmaUnidade) {
```

```
    // Arrange
```

```
    Numero_reserva_modelo modelo;
```

```
    // Act
```

```
    modelo.incrementar();
```

```
    // Assert
```

```
    EXPECT_EQ(modelo.getNumeroReserva(), 2);
```

```
}
```

Teste de Chamadas Consecutivas de Incremento

Garante a estabilidade e a linearidade do acumulador matemático em cenários de execução consecutiva, simulando a criação em massa de múltiplos registos sequenciais.

```

TEST(NumeroReservaTest, Incrementar varias Vezes) {

    // Arrange

    Numero_reserva_modelo modelo;

    // Act

    modelo.incrementar();

    modelo.incrementar();

    modelo.incrementar();

    // Assert

    EXPECT_EQ(modelo.getNumeroReserva(), 4);

}

```

Teste de Independência Estatística de Objetos

Comprova o princípio do isolamento de estado na programação orientada a objetos. Verifica se o incremento do contador de um objeto não altera ou corrompe o número de reserva guardado noutra instância autónoma em memória.

```

TEST(NumeroReservaTest, InstanciasSaoIndependentes) {

    // Arrange

    Numero_reserva_modelo modelo1;

    Numero_reserva_modelo modelo2;

    // Act

    modelo1.incrementar();

    modelo1.incrementar();

    // Assert

    EXPECT_EQ(modelo1.getNumeroReserva(), 3);

    EXPECT_EQ(modelo2.getNumeroReserva(), 1); // não
    afetado

}

```

Teste de Estado Inicial do Fluxo de Reserva

Garante que o estado numérico lógico por omissão de qualquer nova reserva criada no ecossistema assume o valor correspondente à condição de pendente.

```
TEST(AltEstadosTest, EstadoInicialEhPendente) {  
  
    // Arrange  
  
    AltEstadosModel model;  
  
    // Act  
  
    unsigned char estado = model.getEstado();  
  
    // Assert  
  
    EXPECT_EQ(estado, 0);  
  
}
```

Teste de Tradução Textual Inicial do Estado

Avalia se a conversão do código binário interno do estado por omissão corresponde exatamente à string de caracteres "Pendente", assegurando exatidão visual na consola para o utilizador.

```
TEST(AltEstadosTest, EstadoInicialStrEhPendente) {  
  
    // Arrange  
  
    AltEstadosModel model;  
  
    // Act  
  
    std::string estadoStr = model.getEstadoStr();  
  
    // Assert  
  
    EXPECT_EQ(estadoStr, "Pendente");  
  
}
```

Teste de Transição para o Estado Confirmado

Testa o método de alteração dinâmica do estado operacional para confirmado, certificando que o sistema modifica o código lógico e atualiza a representação textual para "Confirmada".

```
TEST(AltEstadosTest,  
ConfirmarAlteraEstadoParaConfirmada) {
```

```

// Arrange

AltEstadosModel model;

// Act

model.confirmar();

// Assert

EXPECT_EQ(model.getEstado(), 1);

EXPECT_EQ(model.getEstadoStr(), "Confirmada");

}

```

Teste de Transição para o Estado Cancelado

Valida a operação de encerramento da viagem, garantindo que o método de cancelamento modifica com sucesso o estado lógico do objeto para o código de cancelamento e atualiza a string para "Cancelada".

```

TEST(AltEstadosTest, CancelarAlteraEstadoParaCancelada)
{

// Arrange

AltEstadosModel model;

// Act

model.cancelar();

// Assert

EXPECT_EQ(model.getEstado(), 2);

EXPECT_EQ(model.getEstadoStr(), "Cancelada");

}

```

Teste de Sequência e Ciclo de Vida do Estado

Simula um cenário real de execução interativa onde uma reserva sofre múltiplas transições de estado consecutivas (Pendente $\rightarrow$ Confirmada $\rightarrow$ Cancelada). Garante que a máquina de estados aceita reconfigurações e guarda o estado final sem perdas de consistência.

```

TEST(AltEstadosTest, ConfirmarDepoisCancelar) {

// Arrange

```

```

AltEstadosModel model;

// Act

model.confirmar();

model.cancelar();

// Assert

EXPECT_EQ(model.getEstado(), 2);

EXPECT_EQ(model.getEstadoStr(), "Cancelada");
}

```

13. Tabelas de taxa de sucesso dos testes

Teste	Percentagem	Notas
Construtores por Omissão	100%	-----
Construtores Parametrizados	100%	-----
Métodos Getters e Setters Gerais	100%	-----
Operador de Igualdade	100%	-----
Modificação de Listas de Viajantes	100%	-----
Regra de Maioridade / Idade Mínima	100%	-----
Gestão de Experiências e Horários Internos	100%	-----
Processamento de Linhas Válidas / Parsing	100%	-----
Tratamento de Exceções para Dados Corrompidos	100%	-----
Tolerância a Ficheiros Inexistentes	100%	-----

Controlo de Ponteiros no Roteiro	100%	-----
Exportação e Persistência de Reservas	100%	-----
Somatório Polimórfico e Acumulação Financeira	100%	-----
Cálculo de IVA e Custo Total Final	100%	-----
Configuração de Métodos de Pagamento	100%	-----
Consistência Cronológica das Datas	100%	-----
Restrições do Calendário Gregoriano	100%	-----
Algoritmo de Anos Bissextos	100%	-----
Cálculo de Distância Temporal / Duração	100%	-----
Manipulação de Datas via Setters	100%	-----
Estado Inicial de Títulos e Notas	100%	-----
Limites de Caracteres do Título da Reserva	100%	-----
Limites de Caracteres das Notas da Viagem	100%	-----
Número de Reserva e Máquina de Estados	100%	-----

Tab. 1. Taxa de Sucesso dos Testes

13. Conclusão

O desenvolvimento do Sistema de Gestão de Planos de Viagem (SGPV) permitiu consolidar de forma prática e integrada os conceitos avançados de Engenharia de Software e Programação Orientada a Objetos (POO) preconizados nesta unidade curricular. Através da linguagem C++, o projeto respondeu com eficácia aos desafios inerentes à digitalização do ecossistema turístico moderno, resultando numa aplicação robusta que otimiza e automatiza com rigor o ciclo de vida do planeamento de viagens. A arquitetura de software adotada, assente no padrão Model-View-Controller (MVC), provou ser uma escolha fundamental para assegurar a separação clara entre a interface de utilizador, a lógica de negócio e o controlo de fluxo. Esta separação de interesses foi substancialmente potenciada pela aplicação dos princípios SOLID e dos padrões GRASP. A distribuição da lógica por múltiplos controladores especializados permitiu mitigar o anti-padrão God Object, reduzindo o acoplamento global e facilitando a manutenibilidade do código. Estruturalmente, o uso estratégico de herança e polimorfismo dinâmico — visível na hierarquia da classe abstrata Localidade (especializada em País, Cidade e Atividade) e na modelação das entidades biográficas em torno de Pessoa — conferiu ao sistema uma flexibilidade e elegância ímpares na manipulação uniforme do catálogo e dos perfis de viajantes.

Do ponto de vista operacional e da integridade de dados, a aplicação evidenciou uma forte resiliência face a cenários imprevistos. O sistema de gestão centralizada de exceções — estruturado a partir da classe nativa `std::exception` e desdobrado em especializações como `InvalidDataException`, `InvalidDateException` e `InvalidTypeException` — garantiu que erros de input ou inconsistências temporais fossem capturados ativamente, salvaguardando a estabilidade da execução standalone em memória. Adicionalmente, as rotinas de persistência local em ficheiros de texto (.txt) demonstraram eficácia na separação entre dados de configuração (catálogo imutável) e dados de sessão, enquanto o recurso a classes de fabrico puro automatizou com precisão o processamento financeiro de custos e o cálculo de IVA das faturas geradas.

Por fim, a exatidão técnica e o sucesso da solução proposta foram formalmente validados através de uma rigorosa bateria de testes unitários. A obtenção de uma taxa de sucesso de 100% em todos os módulos avaliados — incluindo construtores, controlo de ponteiros, tolerância a ficheiros inexistentes, parsing de dados corrompidos e acumulação financeira polimórfica — comprova que o software cumpre escrupulosamente os requisitos de qualidade e correção exigidos.

Como perspetivas de trabalho futuro, embora a arquitetura local garanta excelente portabilidade e execução síncrona livre de atrasos de rede, o sistema beneficiaria de uma evolução para um ambiente distribuído com persistência em bases de dados relacionais e da migração da interface em linha de comandos (consola) para uma Interface Gráfica de Utilizador (GUI) multiplataforma. Em suma, o Grupo 5 cumpriu com distinção todos os objetivos delineados, entregando uma solução que espelha o

rigor metodológico e a relevância prática das boas práticas de arquitetura e design no desenvolvimento de software real.